VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

PRATHEEK P REDDY (1WA24CS215)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by PRATHEEK P REDDY (1WA24CS215), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of an OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one)
a) FCFS
b) SJF
c) Priority
d) Round Robin (Experiment with different quantum sizes for RR algorithm)

FCFS

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
   int n, i;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   int at[n], bt[n], ct[n], tat[n], wt[n];

   printf("Enter Arrival Time:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &at[i]);

   printf("Enter Burst Time:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &bt[i]);

   int t = 0;
```

```
    for(i = 0; i < n; i++)
    {
        if(t < at[i])
            t = at[i];

        ct[i] = t + bt[i];
        t = ct[i];

        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    return 0;
}
```

Result:

```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter number of processes: 4
Enter Arrival Time:
0 0 0 0
Enter Burst Time:
7 3 4 6

Process AT      BT      CT      TAT     WT
P1      0       7       7       7       0
P2      0       3       10      10      7
P3      0       4       14      14      10
P4      0       6       20      20      14
```

Date __/__/____
Page _____

- Program : 1

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS

```
#include <stdio.h>

int main() {
int n, i;
printf("Enter number of processes: ");
scanf("%d", &n);

int at[n], bt[n], ct[n], tat[n], wt[n];

printf("Enter Arrival Time:\n");
for(i=0; i<n; i++)
    scanf("%d", &at[i]);

printf("Enter Burnst Time:\n");
for(i=0; i<n; i++)
    scanf("%d", &bt[i]);

int t=0

for(i=0; i<n; i++)
{
    if(t < at[i])
        t = at[i];
```

```
        ct[i] = t + bt[i];
        t = ct[i];

        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }

    printf("\n Process\tAT\tBT\tCT\tTAT
            \tWT\n");
    for(i=0; i<n; i++)
    {
        printf("P%d\t%d\t%d\t
            %d\t%d\t%d\n");
            i+1, at[i], bt[i], ct[i],
            tat[i], wt[i]);
    }
    return 0;
}
```

OUTPUT

```
Enter number of processes: 4
Enter Arrival Time:
0 0 0 0
Enter Burst Time:
7 3 4 6
```

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	0	3	10	10	7
P3	0	4	14	14	10
P4	0	6	20	20	14

SJF (Non – Preemptive)

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
   int n, i;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   int at[n], bt[n], ct[n], tat[n], wt[n];
   int done[n];

   printf("Enter Arrival Time:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &at[i]);
      done[i] = 0;
   }

   printf("Enter Burst Time:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &bt[i]);

   int t = 0, completed = 0;

   while(completed < n)
   {
      int min = 9999;
```

```
    int index = -1;

    for(i = 0; i < n; i++)
    {
        if(at[i] <= t && done[i] == 0)
        {
            if(bt[i] < min)
            {
                min = bt[i];
                index = i;
            }
        }
    }

    if(index == -1)
    {
        t++;
    }
    else
    {
        ct[index] = t + bt[index];
        t = ct[index];

        tat[index] = ct[index] - at[index];
        wt[index] = tat[index] - bt[index];

        done[index] = 1;
        completed++;
    }
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
```

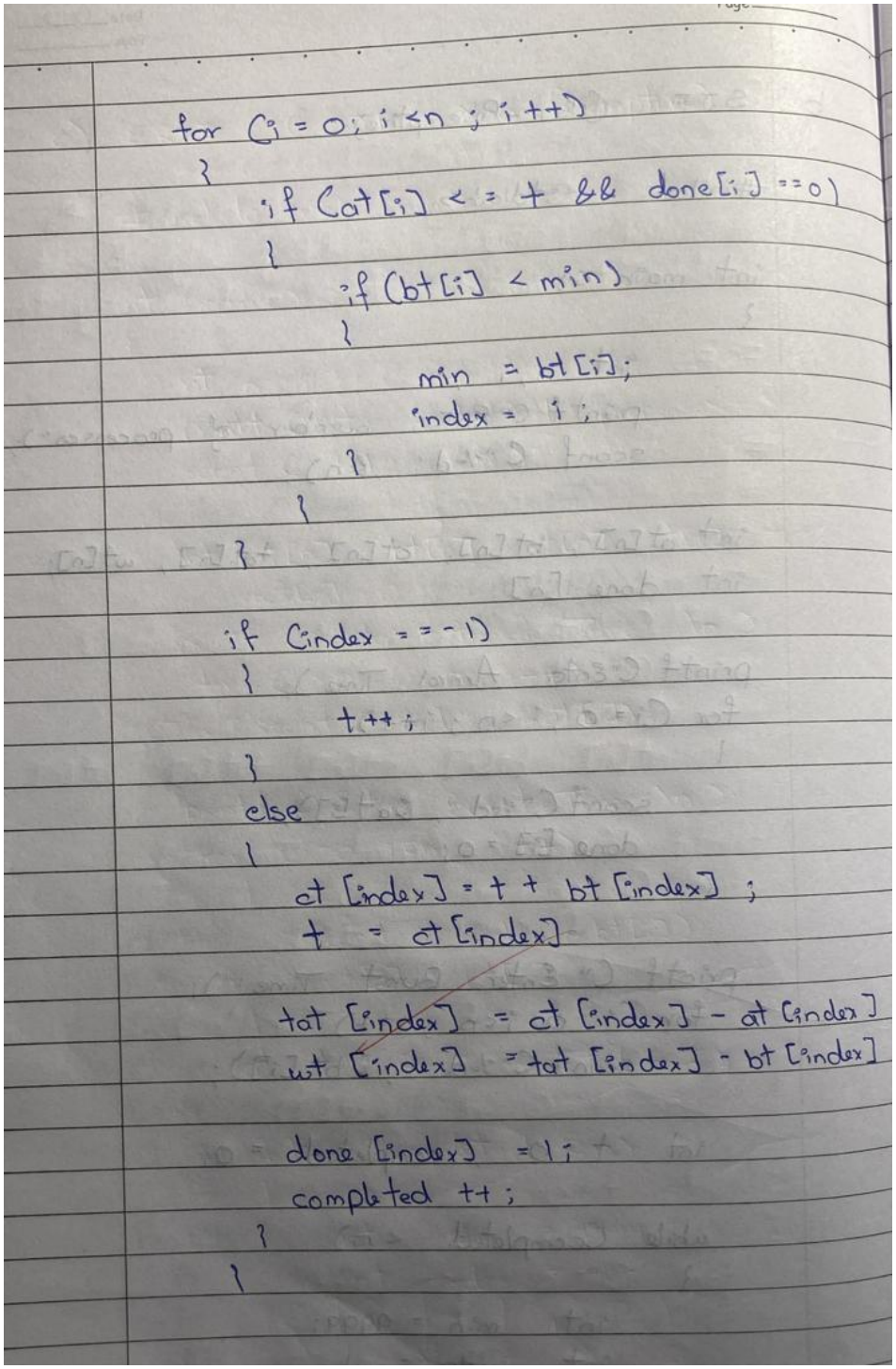
```
    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    return 0;
}
```

Result:

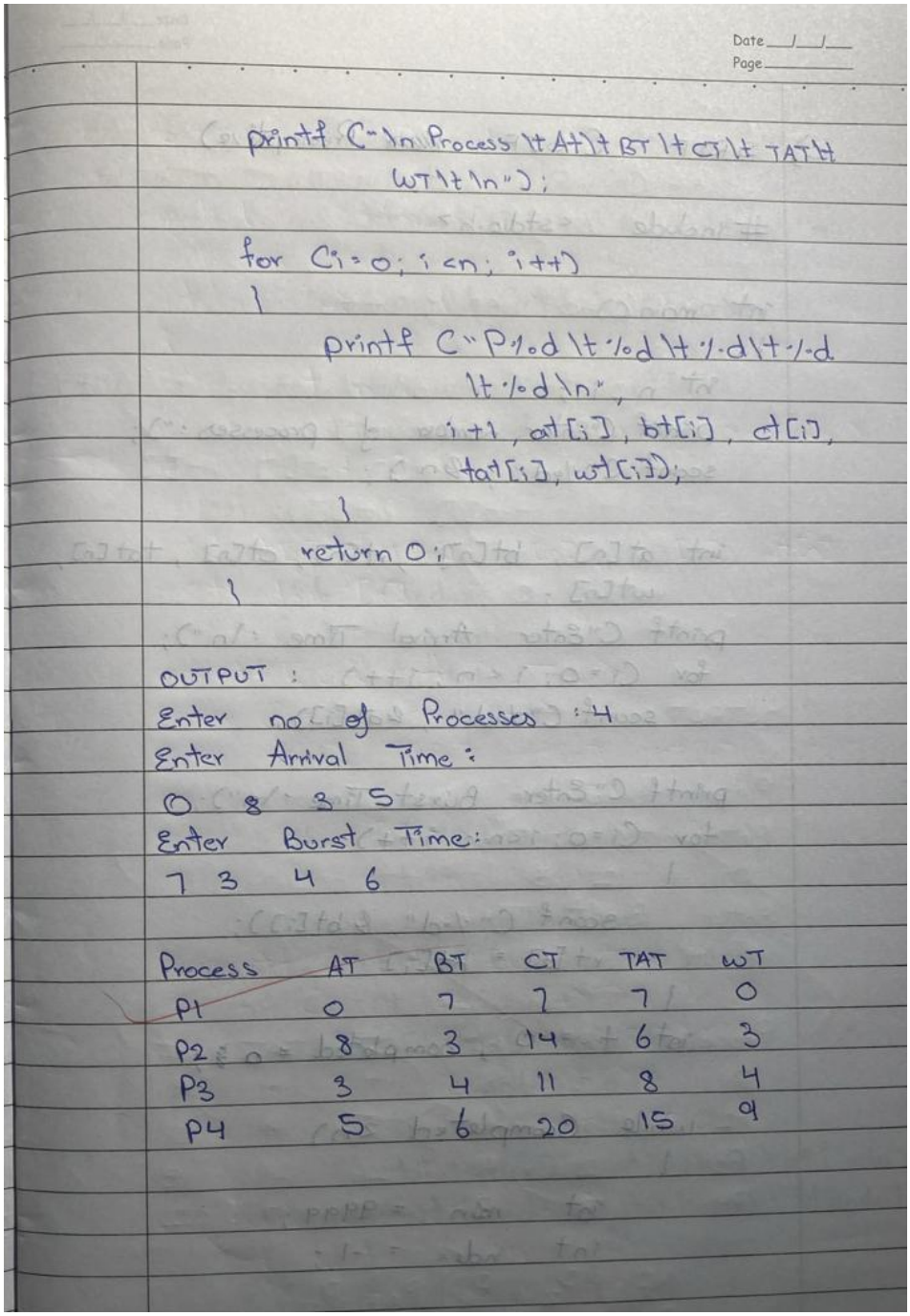
```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter number of processes: 4
Enter Arrival Time:
0 8 3 5
Enter Burst Time:
7 3 4 6

Process AT      BT      CT      TAT     WT
P1      0       7       7       7       0
P2      8       3       14      6       3
P3      3       4       11      8       4
P4      5       6       20      15      9
```

b SJF (Non - Preemptive)

```
#include <stdio.h>

int main ()
{
    int n, i ;
    printf ("Enter number of processes");
    scanf ("%d", &n);

int at[n], bt[n], ct[n], tat[n], wt[n];
int done [n]

printf ("Enter Arrival Time");
for (i = 0 ; i <n ; i++)
{
    scanf ("%d", &at[i]);
    done [i] = 0;
}

printf ("Enter Burst Time:");
for (i = 0; i<n ; i++)
    scanf ("%d", &bt [i]);

int t = 0 , completed = 0;

while (completed <n)
{
    int min = 9999;
    int index = -1;
```



```
for (i = 0; i <n ; i++)
 }
   if (at[i] <= t && done[i] ==0)
   {
      if (bt[i] < min)
      {
         min = bt[i];
         index = i;
      }
   }
}

if (index ==-1)
 }
     t++;
 }
 else
 {
    at [index] = t + bt [index] ;
    t = at [index]

   tat [index] = at [index] - at [index]
   wt [index] = tat [index] - bt [index]

   done [index] = 1;
   completed ++;
  }
}
```



```
printf("\n Process \t AT \t BT \t CT \t TAT \t
        WT \t \n");

for (i=0; i<n; i++)
{
    printf("P%d \t %d \t %d \t %d \t %d
        \t %d \n",
        i+1, at[i], bt[i], ct[i],
        tat[i], wt[i]);
    }
return 0;
}
```

OUTPUT :

Enter no of Processes : 4

Enter Arrival Time :

0 8 3 5

Enter Burst Time:

7 3 4 6

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	8	3	14	6	3
P3	3	4	11	8	4
P4	5	6	20	15	9

SRTF

CODE:

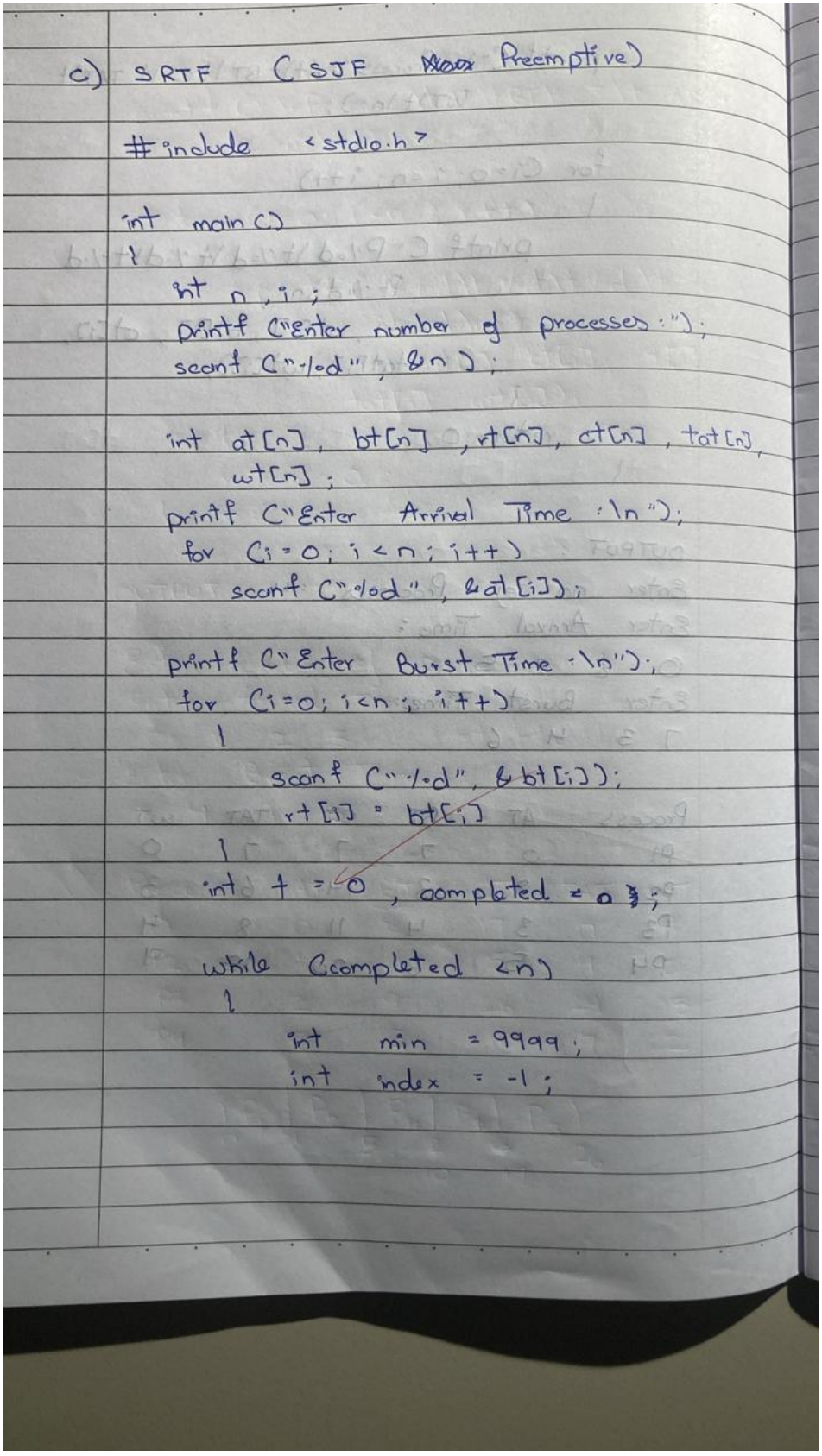
```
#include <stdio.h>

int main()
{
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
   int n, i;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];

   printf("Enter Arrival Time:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &at[i]);

   printf("Enter Burst Time:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &bt[i]);
      rt[i] = bt[i];
   }

   int t = 0, completed = 0;

   while(completed < n)
   {
      int min = 9999;
      int index = -1;
```

```
    for(i = 0; i < n; i++)
    {
        if(at[i] <= t && rt[i] > 0)
        {
            if(rt[i] < min)
            {
                min = rt[i];
                index = i;
            }
        }
    }

    if(index == -1)
    {
        t++;
    }
    else
    {
        rt[index]--;
        t++;

        if(rt[index] == 0)
        {
            ct[index] = t;
            completed++;

            tat[index] = ct[index] - at[index];
            wt[index] = tat[index] - bt[index];
        }
    }
}
```

```
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    return 0;
}
```

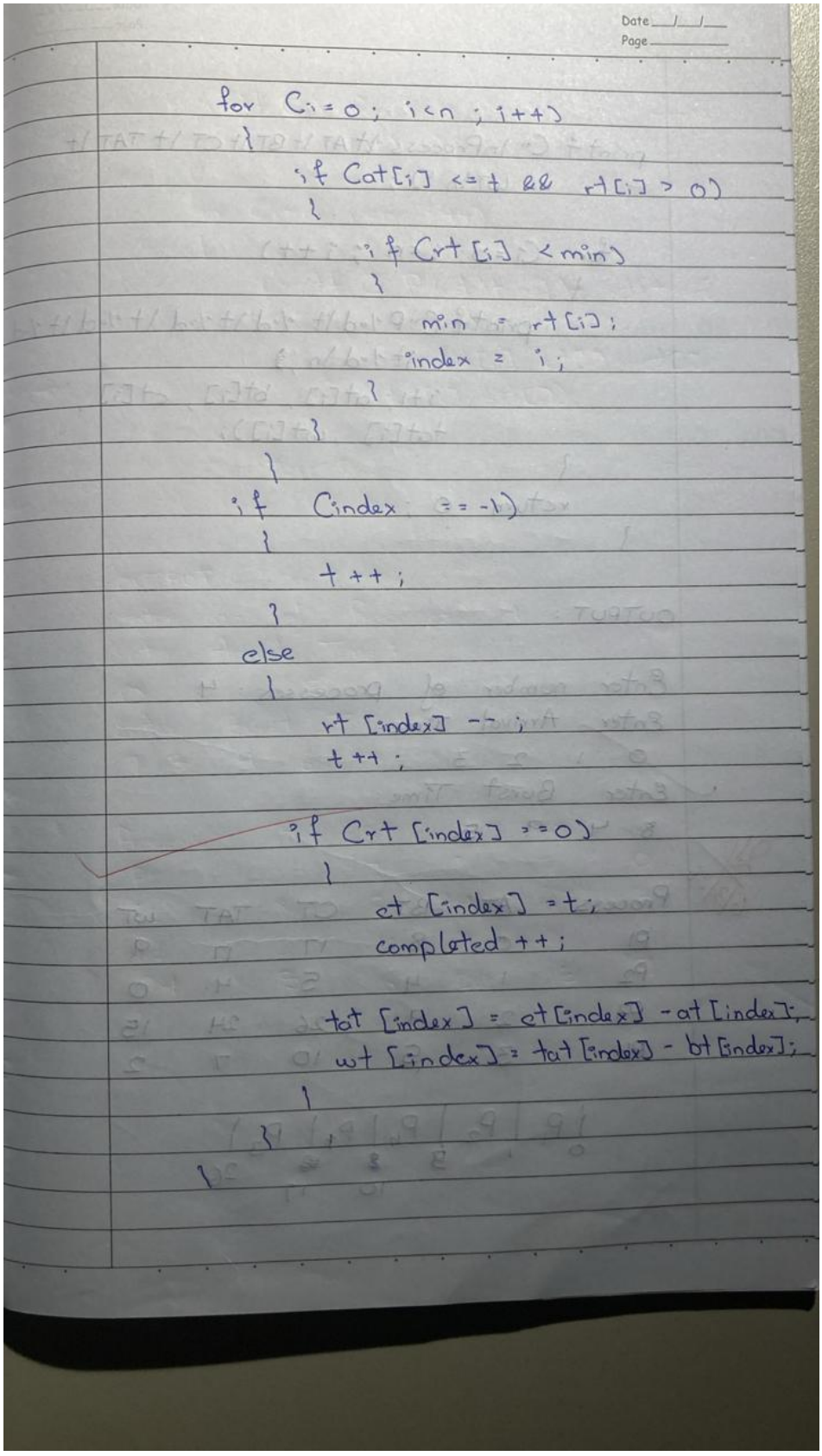
Result:

```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter number of processes: 4
Enter Arrival Time:
0 1 2 3
Enter Burst Time:
8 4 9 5

Process AT      BT      CT      TAT     WT
P1      0       8       17      17      9
P2      1       4       5       4       0
P3      2       9       26      24      15
P4      3       5       10      7       2
```

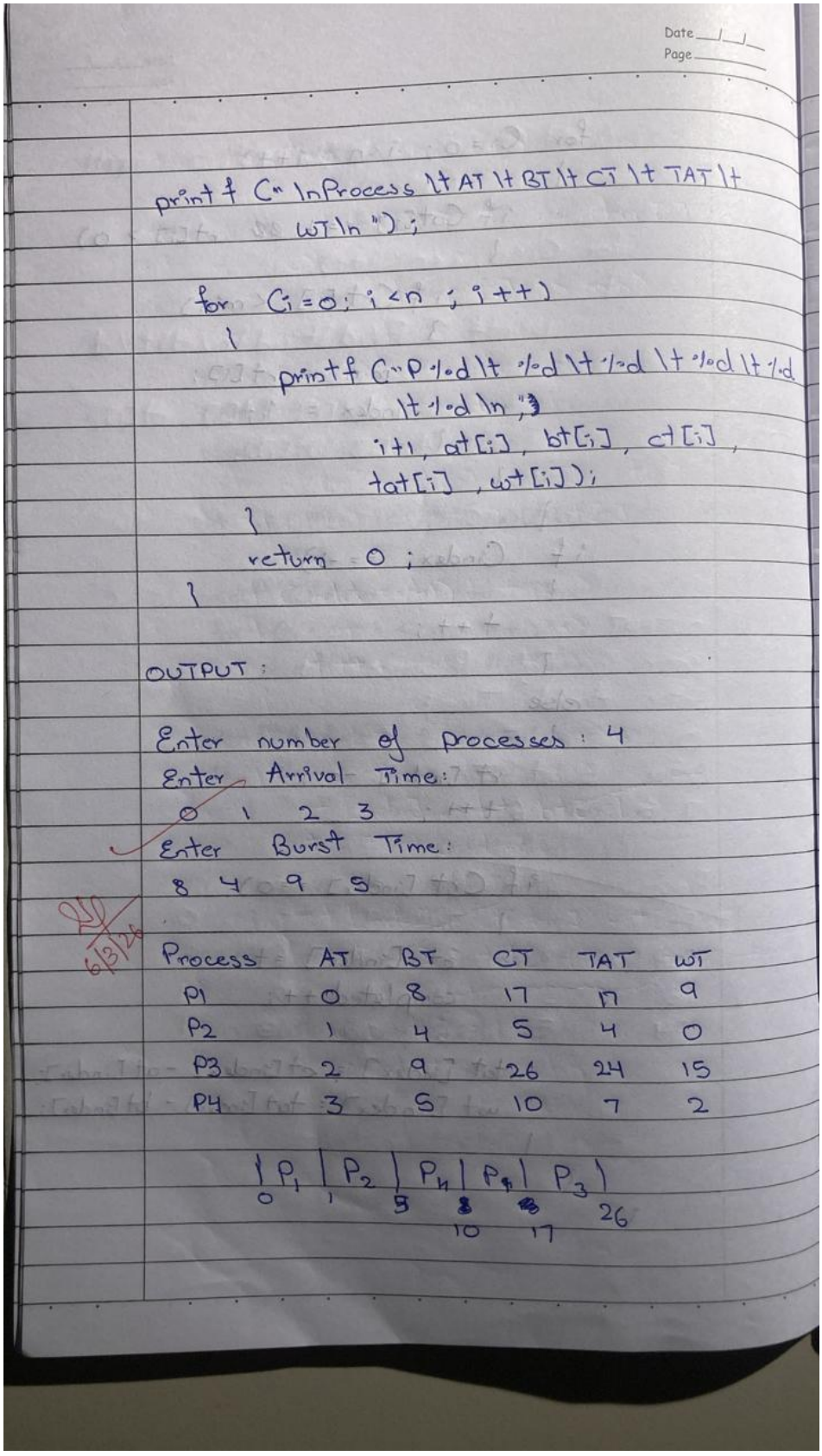


c) SRTF (SJF Preemptive)

```
#include <stdio.h>

int main()
{
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], rt[n], ct[n], tat[n],
        wt[n];
    printf("Enter Arrival Time :\n");
    for(i=0; i<n; i++)
        scanf("%d", &at[i]);

    printf("Enter Burst Time :\n");
    for(i=0; i<n; i++)
    {
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    int t = 0, completed = 0;

    while(completed<n)
    {
        int min = 9999;
        int index = -1;
```



```
for (i = 0; i<n ; i++)
{
    if (at[i] <= t && rt[i] > 0)
    {
        if (rt[i] < min)
        {
            min = rt[i];
            index = i;
        }
    }
}
if (index == -1)
{
    t++;
}
else
{
    rt[index] --;
    t++;

    if (rt[index] == 0)
    {
        et[index] = t;
        completed++;

        tat[index] = et[index] - at[index];
        wt[index] = tat[index] - bt[index];
    }
}
```



```
printf("\nProcess \t AT \t BT \t CT \t TAT \t
        WT \n");

for (i=0; i<n ; i++)
  {
    printf("P%d \t %d \t %d \t %d \t %d \t %d
          \t %d \n",
          i+1, at[i], bt[i], ct[i],
          tat[i] , wt[i]);
  }
  return 0;
}
```

OUTPUT:

Enter number of processes : 4
Enter Arrival Time:
0 1 2 3
Enter Burst Time:
8 4 9 5

6/3/26

Process	AT	BT	CT	TAT	WT
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

P1	P2	P4	P1	P3
0	1	5	10	17	26

PRIORITY NON-PREEMPTIVE

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
   int n, i;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   int at[n], bt[n], pr[n], ct[n], tat[n], wt[n];
   int done[n];

   printf("Enter Arrival Time:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &at[i]);
      done[i] = 0;
   }

   printf("Enter Burst Time:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &bt[i]);

   printf("Enter Priority:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &pr[i]);

   int t = 0, completed = 0;
```

```
while(completed < n)
{
   int min = 9999;
   int index = -1;

   for(i = 0; i < n; i++)
   {
      if(at[i] <= t && done[i] == 0)
      {
         if(pr[i] < min)
         {
            min = pr[i];
            index = i;
         }
      }
   }

   if(index == -1)
   {
      t++;
   }
   else
   {
      ct[index] = t + bt[index];
      t = ct[index];

      tat[index] = ct[index] - at[index];
      wt[index] = tat[index] - bt[index];

      done[index] = 1;
      completed++;
   }
```

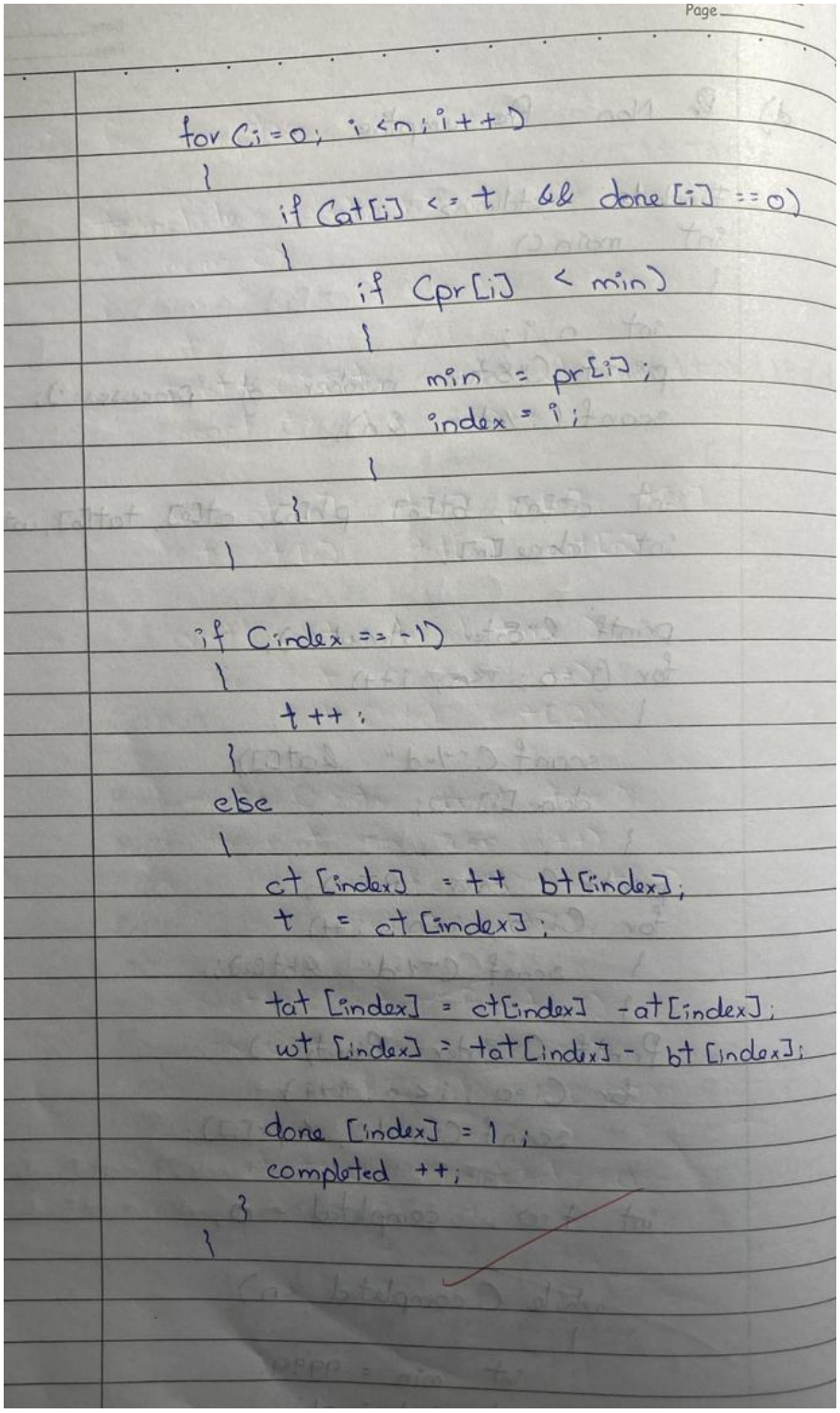
```
    }

    printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
    }

    return 0;
}
```

Result:

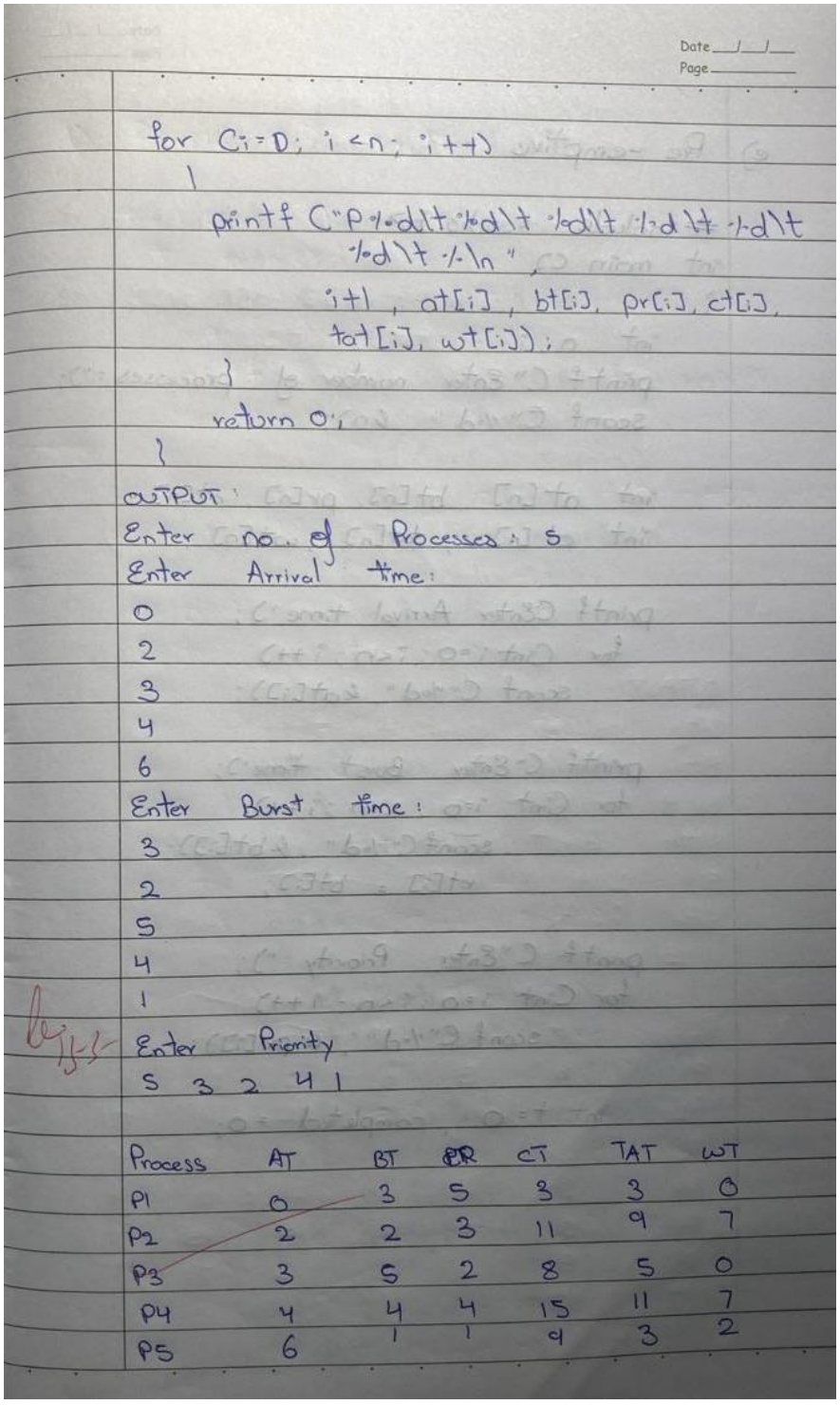
```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter number of processes: 5
Enter Arrival Time:
0
2
3
4
6
Enter Burst Time:
3 2 5 4 1
Enter Priority:
5 3 2 4 1

Process AT      BT      PR      CT      TAT     WT
P1      0       3       5       3       3       0
P2      2       2       3       11      9       7
P3      3       5       2       8       5       0
P4      4       4       4       15      11      7
P5      6       1       1       9       3       2
```

d) Non Pre-emptive

```
#include <stdio.h>
int main()
{
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], ct[n], tat[n], wt[n]
    int done[n];

    printf("Enter Arrival :");
    for (i=0; i<n; i++)
    {
        scanf("%d", &at[i]);
        done[i] = 0;
    }
    printf("Enter BT:");
    for (i=0; i<n; i++)
    {   scanf("%d", &bt[i]);
    }}
    Printf("Enter Priority :");
    for (i=0; i<n; i++)
        scanf("%d", &pr[i]);

    int t=0, completed = 0;

    while (completed <n)
    {
        int min = 9999;
        int index = -1;
```



```
for (i=0; i<n; i++)
{
    if (at[i] <= t && done[i] == 0)
    {
        if (pr[i] < min)
        {
            min = pr[i];
            index = i;
        }
    }
}

if (index == -1)
{
    t++;
}
else
{
    ct[index] = t + bt[index];
    t = ct[index];

    tat[index] = ct[index] - at[index];
    wt[index] = tat[index] - bt[index];

    done[index] = 1;
    completed ++;
}
}
```



```
for (i=0; i<n; i++)
  {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
           i+1, at[i], bt[i], pr[i], ct[i],
           tat[i], wt[i]);
  }
  return 0;
}
```

OUTPUT:

```
Enter no. of Processes: 5
Enter Arrival time:
0
2
3
4
6
Enter Burst time:
3
2
5
4
1
Enter Priority
5 3 2 4 1
```

Process	AT	BT	PR	CT	TAT	WT
P1	0	3	5	3	3	0
P2	2	2	3	11	9	7
P3	3	5	2	8	5	0
P4	4	4	4	15	11	7
P5	6	1	1	9	3	2

PRIORITY PREEMPTIVE

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
   int n, i;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   int at[n], bt[n], pr[n], rt[n];
   int ct[n], tat[n], wt[n];

   printf("Enter Arrival Time:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &at[i]);

   printf("Enter Burst Time:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &bt[i]);
      rt[i] = bt[i];
   }

   printf("Enter Priority:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &pr[i]);

   int t = 0, completed = 0;
```

```
while(completed < n)
{
   int min = 9999;
   int index = -1;

   for(i = 0; i < n; i++)
   {
      if(at[i] <= t && rt[i] > 0)
      {
         if(pr[i] < min)
         {
            min = pr[i];
            index = i;
         }
      }
   }

   if(index == -1)
   {
      t++;
   }
   else
   {
      rt[index]--;
      t++;

      if(rt[index] == 0)
      {
         ct[index] = t;
         tat[index] = ct[index] - at[index];
         wt[index] = tat[index] - bt[index];
```

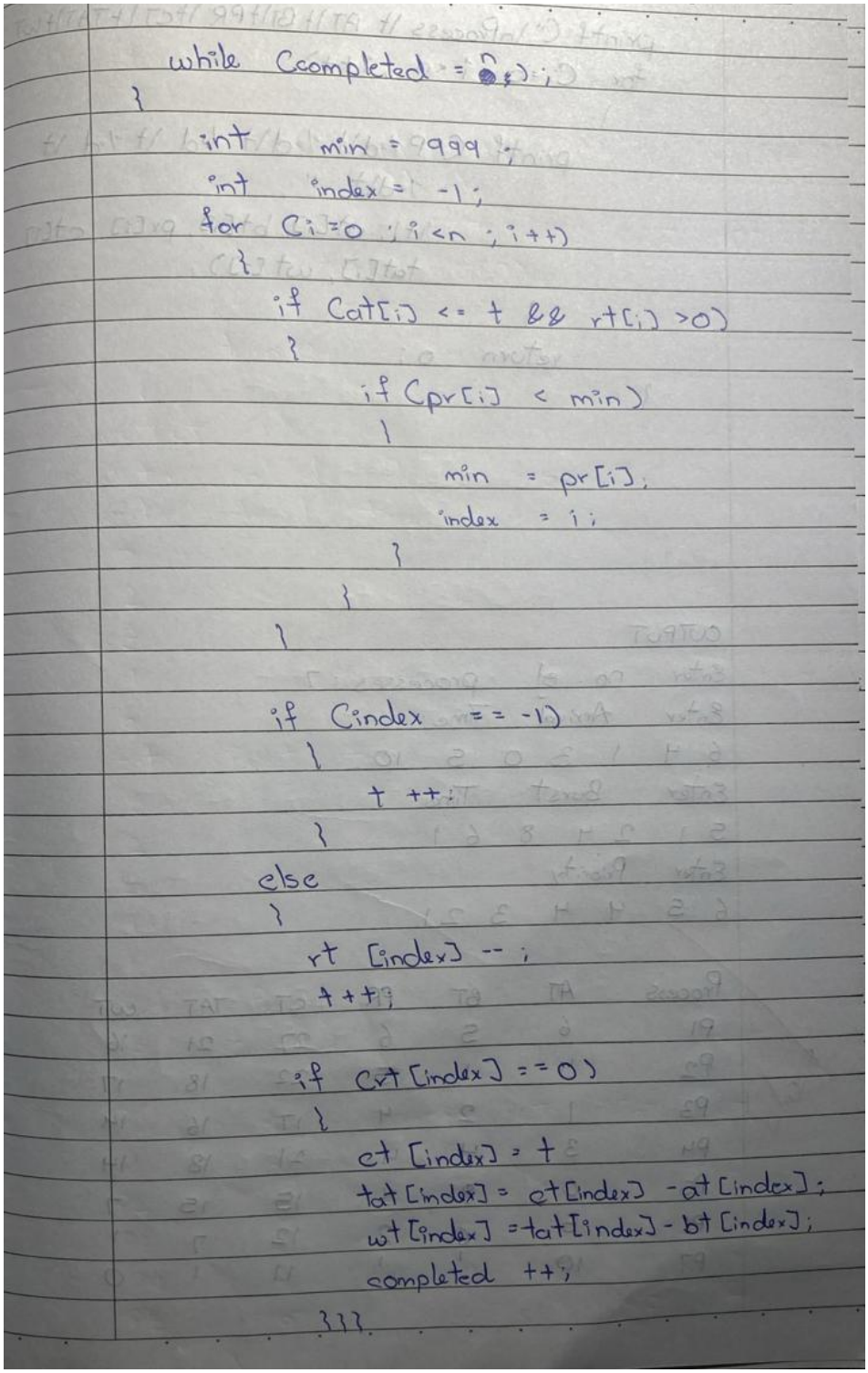
```
            completed++;
        }
    }
}

printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}

return 0;
}
```

Result:

```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter number of processes: 7
Enter Arrival Time:
6 4 1 3 0 5 10
Enter Burst Time:
5 1 2 4 8 6 1
Enter Priority:
6 5 4 4 3 2 1

Process AT      BT      PR      CT      TAT     WT
P1      6       5       6       27      21      16
P2      4       1       5       22      18      17
P3      1       2       4       17      16      14
P4      3       4       4       21      18      14
P5      0       8       3       15      15      7
P6      5       6       2       12      7       1
P7      10      1       1       11      1       0
```

e) Pre -emptive

```
#include <stdio.h>
int main ()
{
    int n, i ;
    printf (" Enter number of processes : ");
    scanf (" %d " , &n);

    int at[n] , bt[n], pr[n], rt[n] ;
    int ct[n] , tat[n] , wt[n] ;

    printf (Enter Arrival time ");
     for (int i =0 ; i<n ; i ++)
         scanf (" %d ", &at[i]);

    printf (" Enter Burst time ");
     for (int i=0 ; i <n ; i++)
              scanf (" %d " , &bt[i]);
               rt[i] = bt[i];

    printf (" Enter Priority : ");
     for (int i =0 ; i<n ; i ++)
          scanf (" %d ", & pr[i]);

      int t = 0 , completed = 0;
```



```
while (completed != n);
}
int min = 999;
int index = -1;
for (i=0 ; i<n ; i++)
{
if (at[i] <= t && rt[i] >0)
{
if (pr[i] < min)
{
min = pr[i];
index = i;
}
}
}
if (index == -1)
{
t ++;
}
else
{
rt [index] --;
t++;
if (rt [index] == 0)
{
et [index] = t
tat [index] = et[index] - at [index];
wt [index] = tat[index] - bt [index];
completed ++;
}}}
```

```
printf("\nProcess \t AT \t BT \t PR \t CT \t TAT \t WT");
for (i=0 ; i<n ; i++)
  {
    printf("P%d \t %d \t %d \t %d \t
          %d \t ",
          i+1, at[i], bt[i], pr[i], ct[i],
          tat[i], wt[i]);
  }
  return 0;
}
```

OUTPUT

```
Enter no. of processes : 7
Enter Arrival Time :
6 4 1 3 0 5 10
Enter Burst Time :
5 1 2 4 8 6 1
Enter Priority
6 5 4 4 3 2 1
```

Process	AT	BT	PR	CT	TAT	WT
P1	6	5	6	27	21	16
P2	4	1	5	22	18	17
P3	1	2	4	17	16	14
P4	3	4	4	21	18	14
P5	0	8	3	15	15	7
P6	5	6	2	12	7	1
P7	10	1	1	11	1	0

ROUND ROBIN

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
   int n, tq, i;

   printf("Enter number of processes: ");
   scanf("%d",&n);

   printf("Enter Time Quantum: ");
   scanf("%d",&tq);

   int pid[n], at[n], bt[n], rt[n];
   int wt[n], tat[n], ct[n], resp[n];
   int queue[100];
   int front=0, rear=0;

   for(i=0;i<n;i++)
   {
      printf("Enter Process ID, Arrival Time, Burst Time:\n");
      scanf("%d %d %d",&pid[i],&at[i],&bt[i]);

      rt[i] = bt[i];
      wt[i] = 0;
      tat[i] = 0;
      resp[i] = -1;
   }
```

```
int current_time = 0;
int completed = 0;
int visited[n];

for(i=0;i<n;i++)
   visited[i] = 0;

while(completed < n)
{
   for(i=0;i<n;i++)
   {
      if(at[i] <= current_time && visited[i] == 0)
      {
         queue[rear++] = i;
         visited[i] = 1;
      }
   }

   if(front == rear)
   {
      current_time++;
      continue;
   }

   int p = queue[front++];

   if(resp[p] == -1)
      resp[p] = current_time - at[p];

   if(rt[p] > tq)
   {
      current_time += tq;
```

```
            rt[p] -= tq;

            for(i=0;i<n;i++)
            {
               if(at[i] <= current_time && visited[i] == 0)
               {
                  queue[rear++] = i;
                  visited[i] = 1;
               }
            }

            queue[rear++] = p;
         }
         else
         {
            current_time += rt[p];
            rt[p] = 0;

            ct[p] = current_time;
            tat[p] = ct[p] - at[p];
            wt[p] = tat[p] - bt[p];
            completed++;
         }
      }

      float avg_wt=0, avg_tat=0;

      printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

      for(i=0;i<n;i++)
      {
         printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
```

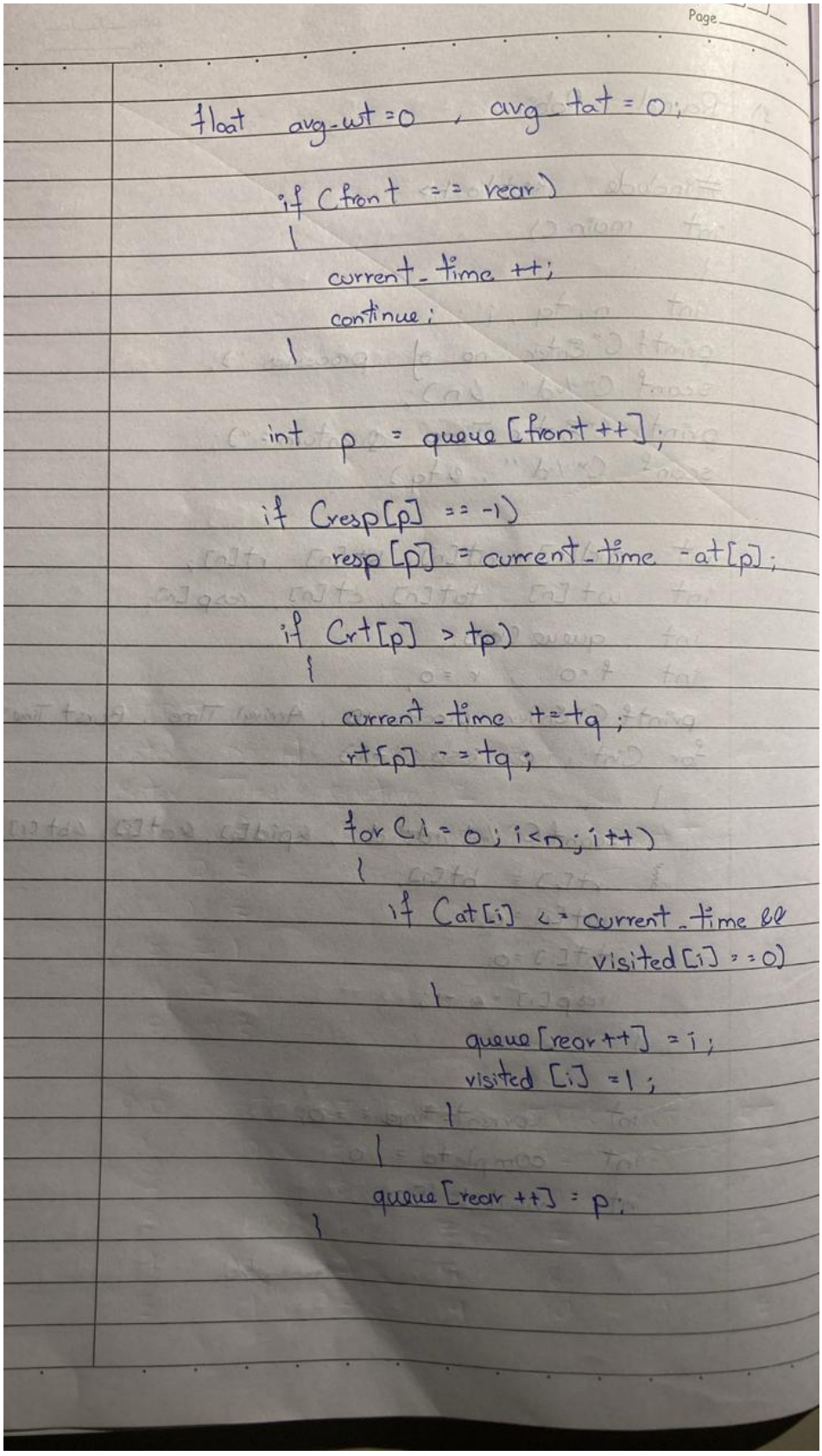
```
        pid[i],at[i],bt[i],ct[i],tat[i],wt[i],resp[i]);

        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nAverage Waiting Time = %.2f",avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);

    return 0;
}
```

Result:

```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter number of processes: 5
Enter Time Quantum: 2
Enter Process ID, Arrival Time, Burst Time:
1 0 5
Enter Process ID, Arrival Time, Burst Time:
2 1 3
Enter Process ID, Arrival Time, Burst Time:
3 2 1
Enter Process ID, Arrival Time, Burst Time:
4 3 2
Enter Process ID, Arrival Time, Burst Time:
5 4 3

PID     AT      BT      CT      TAT     WT      RT
1       0       5       13      13      8       0
2       1       3       12      11      8       1
3       2       1       5       3       2       2
4       3       2       9       6       4       4
5       4       3       14      10      7       5

Average Waiting Time = 5.80
Average Turnaround Time = 8.60
```

f) Round Robin

```
#include <stdio.h>
int main()
{
int n, tq, i;
printf("Enter no. of processes: ");
scanf("%d", &n);
printf("Enter Time Quantum: ");
scanf("%d", &tq);

int pid[n], at[n], bt[n], rt[n];
int wt[n], tat[n], ct[n], resp[n];
int queue[100];
int f=0, r=0;
printf("Enter process ID, Arrival Time, Burst Time");
for(int i=0; i<n; i++)
  {
    scanf("%d %d %d", &pid[i], &at[i], &bt[i]
  }  rt[i] = bt[i];
    wt[i] = 0;
    tat[i] =0;
    resp[i] = -1;
  }

  int current_time = 0;
  int complete = 0;
  int visited[];
```



```
float avg_wt = 0, avg_tat = 0;

if (front == rear)
{
    current_time ++;
    continue;
}

int p = queue[front++];

if (resp[p] == -1)
    resp[p] = current_time - at[p];

if (rt[p] > tq)
{
    current_time += tq;
    rt[p] -= tq;

    for (i = 0; i<n; i++)
    {
        if (at[i] <= current_time &&
                visited[i] == 0)
        {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }
    queue[rear++] = p;
}
```

OUTPUT

Enter no. of processes : 5

Enter Time Quatum : 2

Enter Process ID, Arrival time, Burst Time

1 0 5

2 1 3

3 2 1

4 3 2

5 4 3

PID	AT	BT	CT	TAT	WT	RT
1	0	5	13	13	8	0
2	1	3	12	11	8	1
3	2	1	5	3	2	2
4	3	2	9	6	4	4
5	4	3	14	10	7	5

Program -2

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
   int n, i;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], q[n];

   printf("Enter Arrival Time:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &at[i]);

   printf("Enter Burst Time:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &bt[i]);
      rt[i] = bt[i];
   }

   printf("Enter Queue (1 = RR, 2 = FCFS):\n");
   for(i = 0; i < n; i++)
```

```
    scanf("%d", &q[i]);

int tq;
printf("Enter Time Quantum: ");
scanf("%d", &tq);

int t = 0, completed = 0;

printf("\nGantt Chart:\n| ");

while(completed < n)
{
    int found = 0;


    for(i = 0; i < n; i++)
    {
        if(q[i] == 1 && rt[i] > 0 && at[i] <= t)
        {
            int exec = (rt[i] > tq) ? tq : rt[i];

            for(int k = 0; k < exec; k++)
            {
                printf("P%d ", i+1);
                t++;
            }

            rt[i] -= exec;

            if(rt[i] == 0)
            {
                ct[i] = t;
```

```
            completed++;
        }

        found = 1;
    }
}


if(!found)
{
    for(i = 0; i < n; i++)
    {
        if(q[i] == 2 && rt[i] > 0 && at[i] <= t)
        {

            while(rt[i] > 0)
            {
                // check if any Q1 arrives → break
                int interrupt = 0;
                for(int j = 0; j < n; j++)
                {
                    if(q[j] == 1 && rt[j] > 0 && at[j] <= t)
                    {
                        interrupt = 1;
                        break;
                    }
                }

                if(interrupt) break;

                printf("P%d ", i+1);
                rt[i]--;
```

```
            t++;
        }

        if(rt[i] == 0)
        {
            ct[i] = t;
            completed++;
        }

        found = 1;
      }
    }
  }

  // CPU idle
  if(!found)
  {
    printf("Idle ");
    t++;
  }
}

printf("|\n");

// Calculate TAT & WT
for(i = 0; i < n; i++)
{
  tat[i] = ct[i] - at[i];
  wt[i] = tat[i] - bt[i];
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
```

```
    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    return 0;
}
```

Result:

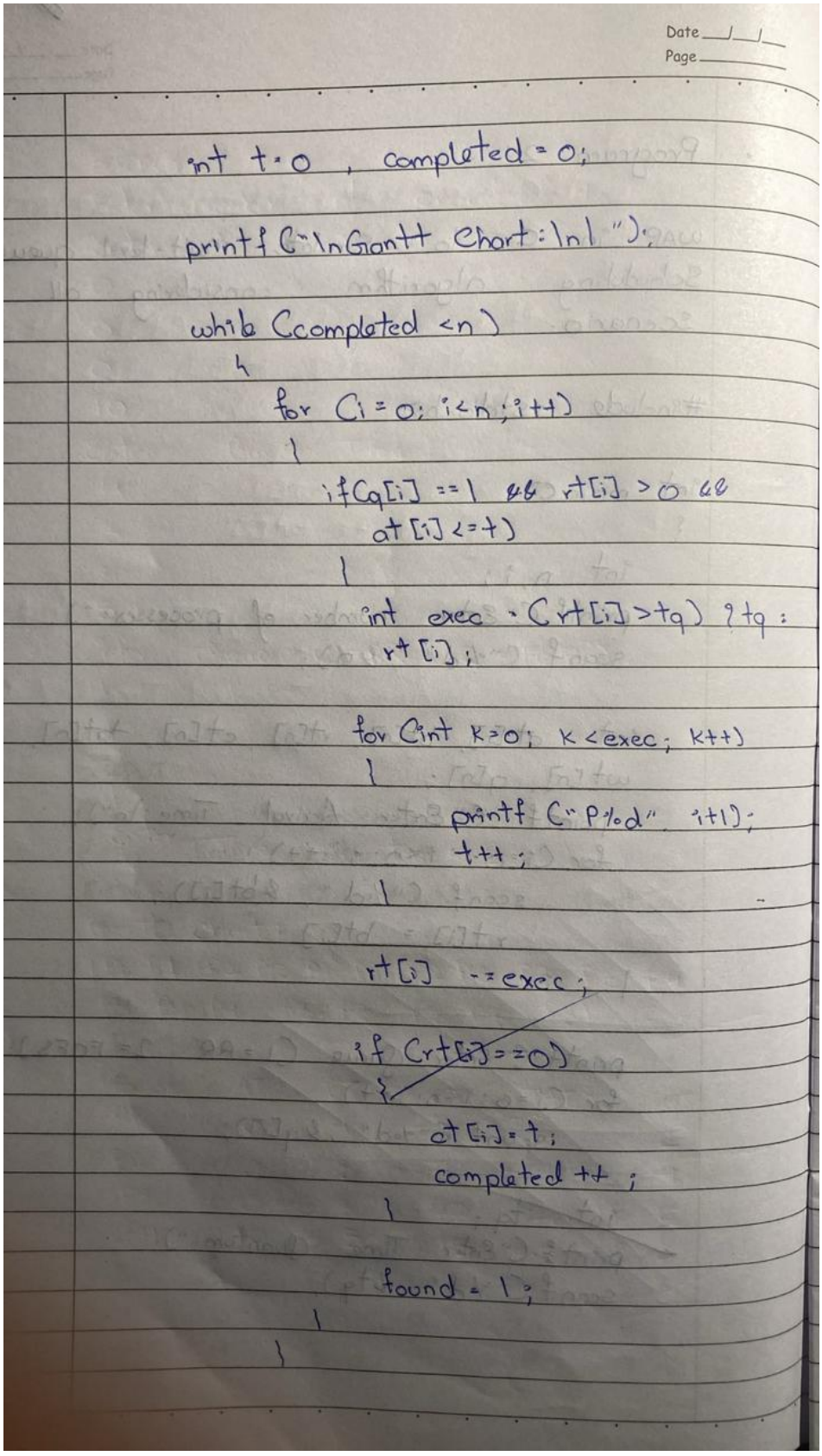
```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter number of processes: 4
Enter Arrival Time:
0 0 0 10
Enter Burst Time:
4 3 8 5
Enter Queue (1 = RR, 2 = FCFS):
1 1 2 1
Enter Time Quantum: 2

Gantt Chart:
| P1 P1 P2 P2 P1 P1 P2 P3 P3 P3 P4 P4 P4 P4 P4 P3 P3 P3 P3 P3 |

Process AT      BT      CT      TAT     WT
P1      0       4       6       6       2
P2      0       3       7       7       4
P3      0       8       20      20      12
P4      10      5       15      5       0
```

Date ___/___/___
Page _____

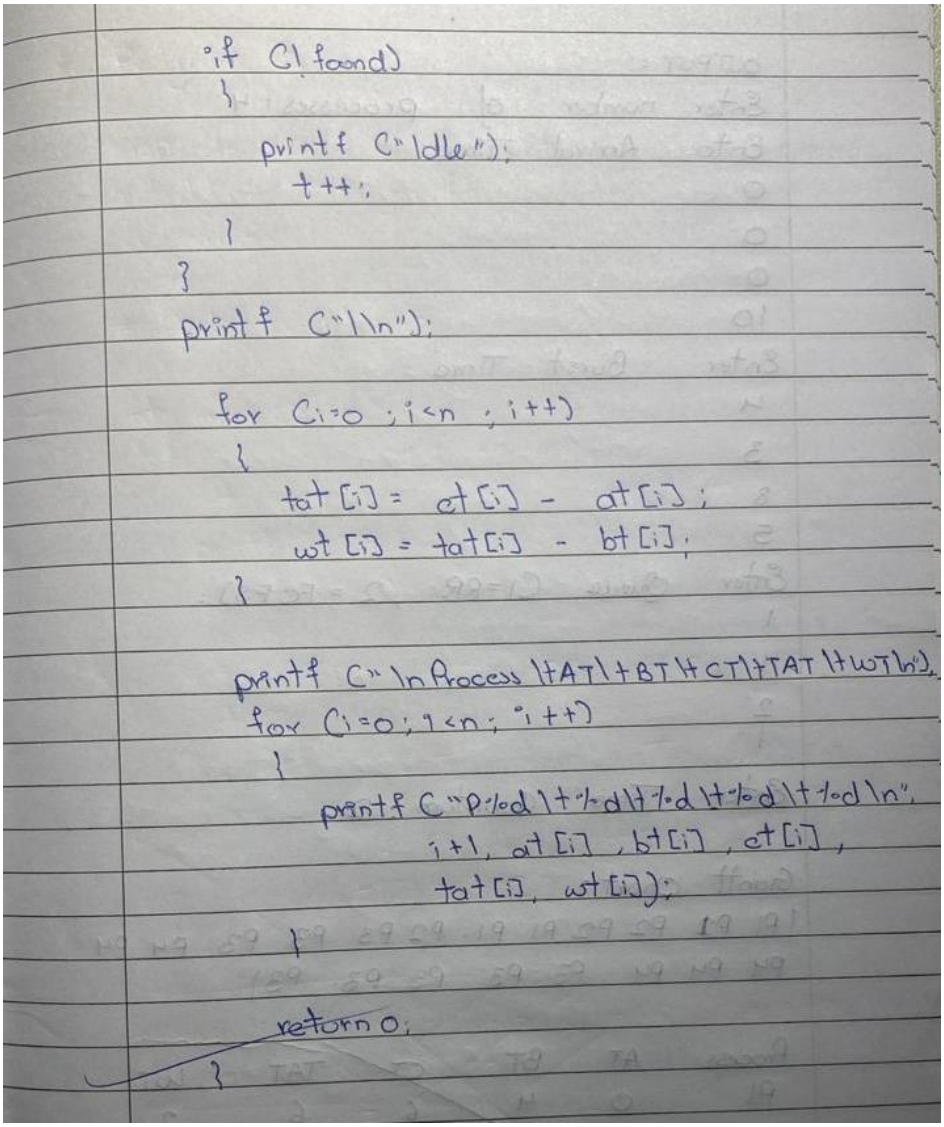
Program 2:

WAP in C to Simulate Multi-level queue Scheduling algorithm considering all scenario.

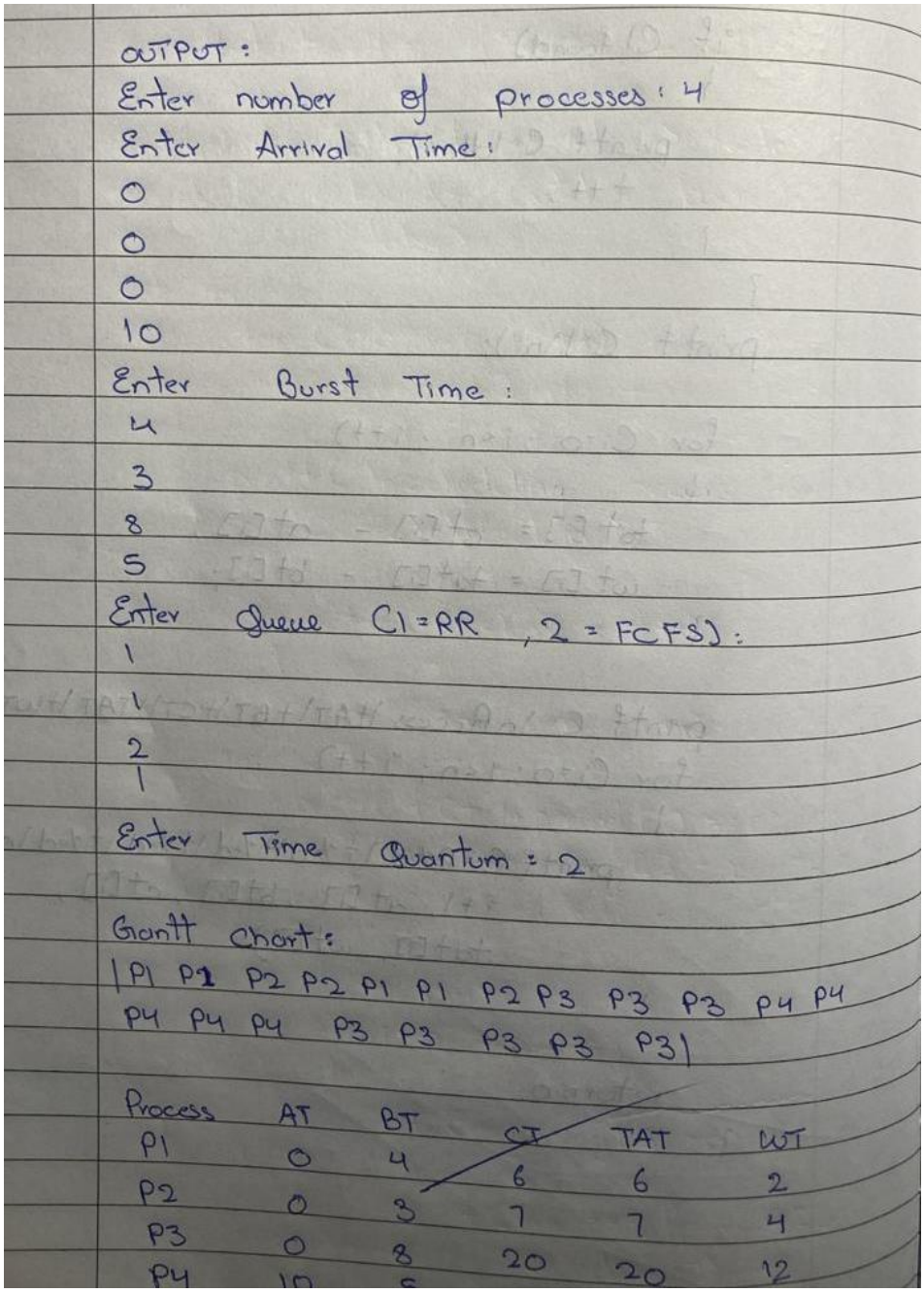
```
#include <stdio.h>

int main()
{
    int n,i;
    printf("Enter number of processes:");
    scanf("%d",&n);

    int at[n], bt[n], rt[n], ct[n], tat[n],
    wt[n], q[n];
    printf("Enter Arrival Time:\n");
    for(i=0; i<n ; i++)
        scanf("%ed", &bt[i]);
        rt[i] = bt[i]
}

printf("Enter Queue (1 = RR, 2 = FCFS):\n");
for(i=0;i<n;i++)
    scanf("%d",&q[i]);

int tq;
printf("Enter Time Quantum:");
scanf("%d",&tq);
```



```
int t=0, completed=0;

printf("\nGantt Chart:\n");

while(completed<n)
{
    for(i=0; i<n; i++)
    {
        if(q[i]==1 && rt[i]>0 &&
           at[i]<=t)
        {
            int exec = (rt[i]>tq) ? tq :
            rt[i];

            for(int k=0; k<exec; k++)
            {
                printf("P%d", i+1);
                t++;
            }

            rt[i] -= exec;

            if(rt[i]==0)
            {
                at[i]=t;
                completed++;
            }

            found=1;
        }
    }
```



```
    if (!found)
    {
        printf ("Idle");
        t++;
    }
}
printf ("\n");

for (i=0 ; i<n ; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
}

printf ("\nProcess \tAT\t BT\t CT\t TAT \t WT\n");
for (i=0; i<n; i++)
{
    printf ("p%d \t %d \t %d \t %d \t %d \t %d \n",
            i+1, at[i], bt[i], ct[i],
            tat[i], wt[i]);
}

return 0;
}
```



OUTPUT :

Enter number of processes : 4

Enter Arrival Time :

0

0

0

10

Enter Burst Time :

4

3

8

5

Enter Queue (1 = RR , 2 = FCFS) :

1

1

2

1

Enter Time Quantum : 2

Gantt chart :

|P1 P1 P2 P2 P1 P1 P2 P3 P3 P3 P4 P4 P4 P4 P4 P3 P3 P3 P3 P3|

Process	AT	BT	CT	TAT	WT
P1	0	4	6	6	2
P2	0	3	7	7	4
P3	0	8	20	20	12
P4	10	5			

Program -3

Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one)
a) Rate- Monotonic
b) Earliest-deadline First
c) Proportional scheduling

RATE – MONOTONIC

CODE:

```
#include <stdio.h>
#include<math.h>

int main()
{
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
   int n, i, j, t = 0, lcm = 1;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   int burst[n], period[n], remaining[n];

   printf("Enter CPU burst times:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &burst[i]);

   printf("Enter time periods:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &period[i]);


   for(i = 0; i < n; i++)
```

```
{
   int max = (lcm > period[i]) ? lcm : period[i];
   while(1)
   {
      if(max % lcm == 0 && max % period[i] == 0)
      {
         lcm = max;
         break;
      }
      max++;
   }
}

printf("\nLCM=%d\n", lcm);

printf("\nRate Monotonic Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for(i = 0; i < n; i++)
   printf("%d\t%d\t%d\n", i+1, burst[i], period[i]);


float util = 0;
for(i = 0; i < n; i++)
   util += (float)burst[i] / period[i];

float bound = n * (pow(2, (float)1/n) - 1);

printf("\n%f <= %f => %s\n", util, bound,
      (util <= bound) ? "true" : "false");


for(i = 0; i < n; i++)
```

```
    remaining[i] = 0;

printf("\nScheduling occurs for %d ms\n\n", lcm);


for(t = 0; t < lcm; t++)
{

    for(i = 0; i < n; i++)
    {
        if(t % period[i] == 0)
            remaining[i] = burst[i];
    }


    int min = -1;
    for(i = 0; i < n; i++)
    {
        if(remaining[i] > 0)
        {
            if(min == -1 || period[i] < period[min])
                min = i;
        }
    }

    if(min != -1)
    {
        printf("%dms onwards: Process %d running\n", t, min+1);
        remaining[min]--;
    }
    else
    {
```

```
            printf("%dms onwards: CPU is idle\n", t);
        }
    }
    return 0;
}
```

Result:

```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter number of processes: 2
Enter CPU burst times:
20 35
Enter time periods:
50 100

LCM=100

Rate Monotonic Scheduling:
PID     Burst   Period
1       20      50
2       35      100

0.750000 <= 0.828427 => true

Scheduling occurs for 100 ms

0ms onwards: Process 1 running
1ms onwards: Process 1 running
2ms onwards: Process 1 running
3ms onwards: Process 1 running
4ms onwards: Process 1 running
```

```
75ms onwards: CPU is idle
76ms onwards: CPU is idle
77ms onwards: CPU is idle
78ms onwards: CPU is idle
79ms onwards: CPU is idle
80ms onwards: CPU is idle
81ms onwards: CPU is idle
82ms onwards: CPU is idle
83ms onwards: CPU is idle
84ms onwards: CPU is idle
85ms onwards: CPU is idle
86ms onwards: CPU is idle
87ms onwards: CPU is idle
88ms onwards: CPU is idle
89ms onwards: CPU is idle
90ms onwards: CPU is idle
91ms onwards: CPU is idle
92ms onwards: CPU is idle
93ms onwards: CPU is idle
94ms onwards: CPU is idle
95ms onwards: CPU is idle
96ms onwards: CPU is idle
97ms onwards: CPU is idle
98ms onwards: CPU is idle
99ms onwards: CPU is idle


...Program finished with exit code 0
Press ENTER to exit console.
```

EARLIEST DEADLINE FIRST

CODE:

```
#include <stdio.h>
#include <limits.h>

int main() {
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
   int n, i, time, hyperperiod = 20;

   printf("Enter number of tasks: ");
   scanf("%d", &n);

   int C[n], D[n], T[n];
   int remaining[n];
   int next_arrival[n];
   int abs_deadline[n];



   for(i = 0; i < n; i++) {
      printf("Enter Capacity, Deadline, Period for Task %d: ", i + 1);
      scanf("%d %d %d", &C[i], &D[i], &T[i]);

      remaining[i] = 0;
      next_arrival[i] = 0;
      abs_deadline[i] = 0;
   }

   printf("\n--- EDF Scheduling (Periodic Tasks) ---\n");

   for(time = 0; time < hyperperiod; time++) {
```

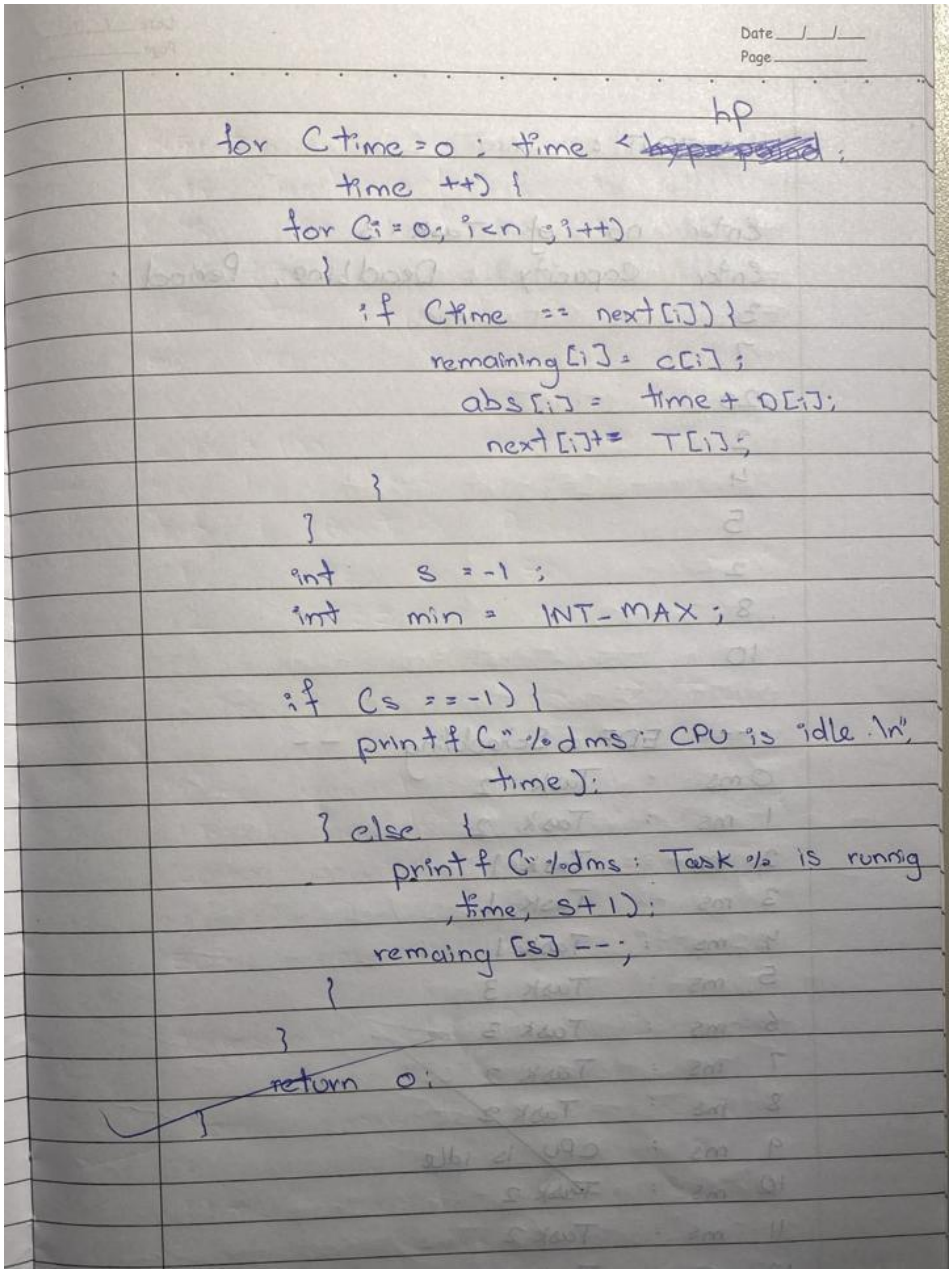
```
        for(i = 0; i < n; i++) {
            if(time == next_arrival[i]) {
                remaining[i] = C[i];
                abs_deadline[i] = time + D[i];
                next_arrival[i] += T[i];
            }
        }

        int selected = -1;
        int min_deadline = INT_MAX;


        for(i = 0; i < n; i++) {
            if(remaining[i] > 0 && abs_deadline[i] < min_deadline) {
                min_deadline = abs_deadline[i];
                selected = i;
            }
        }

        if(selected == -1) {
            printf("%d ms : CPU is idle.\n", time);
        } else {
            printf("%d ms : Task %d is running.\n", time, selected + 1);
            remaining[selected]--;
        }
    }

    return 0;
}
```

Result:

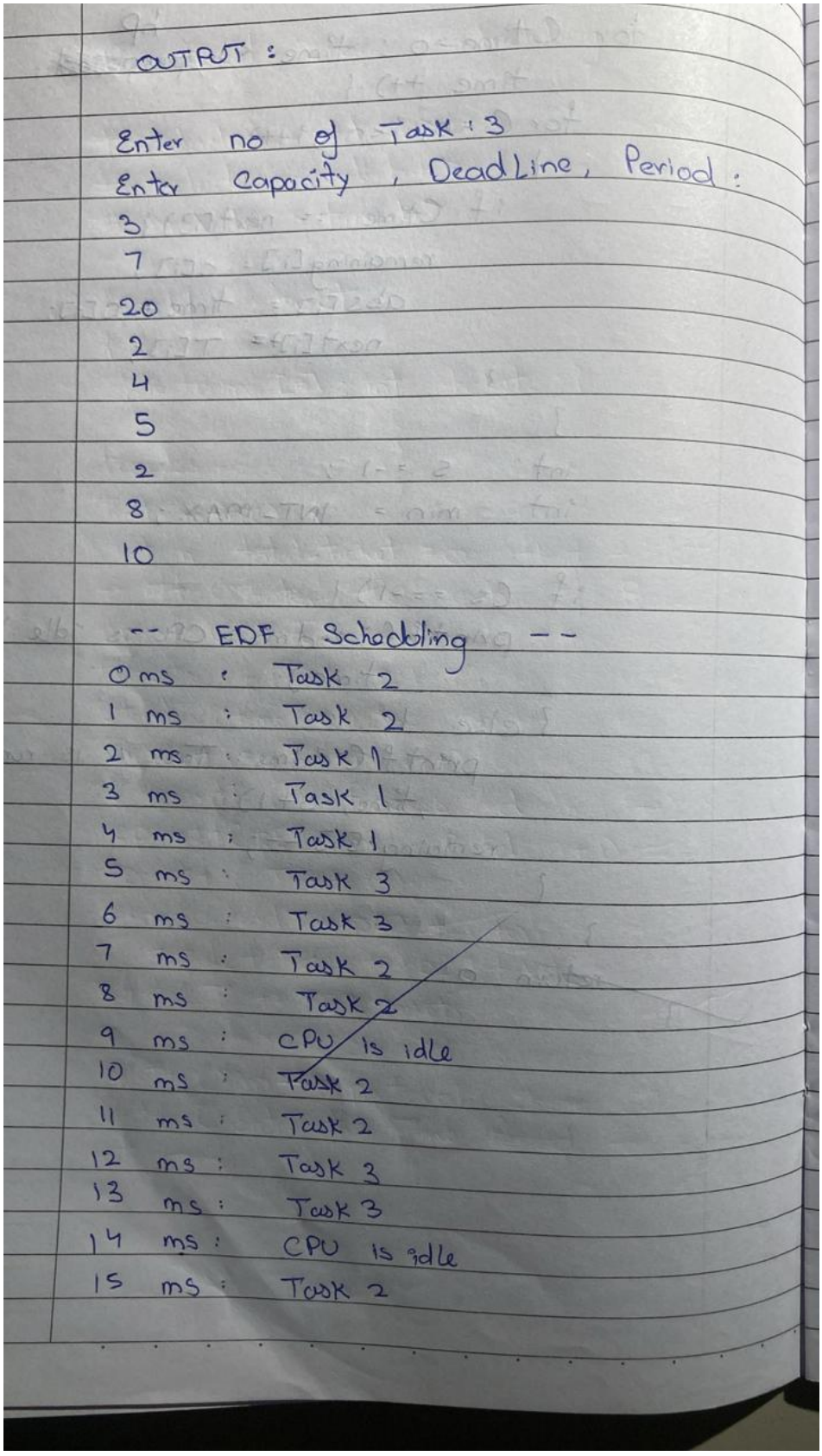
```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter number of tasks: 3
Enter Capacity, Deadline, Period for Task 1: 3 7 20 2 4 5 2 8 10
Enter Capacity, Deadline, Period for Task 2: Enter Capacity, Deadline, Period for Task 3:
--- EDF Scheduling (Periodic Tasks) ---
0 ms : Task 2 is running.
1 ms : Task 2 is running.
2 ms : Task 1 is running.
3 ms : Task 1 is running.
4 ms : Task 1 is running.
5 ms : Task 3 is running.
6 ms : Task 3 is running.
7 ms : Task 2 is running.
8 ms : Task 2 is running.
9 ms : CPU is idle.
10 ms : Task 2 is running.
11 ms : Task 2 is running.
12 ms : Task 3 is running.
13 ms : Task 3 is running.
14 ms : CPU is idle.
15 ms : Task 2 is running.
16 ms : Task 2 is running.
17 ms : CPU is idle.
18 ms : CPU is idle.
19 ms : CPU is idle.


...Program finished with exit code 0
```

2 Earliest Deadline First

```
#include <stdio.h>
#include <stdio.h>

int main() {
    int n, i, time, hp = 20
    printf("Enter no. of task :");
    scanf("%d", &n);

    int C[n], D[n], T[n];
    int remaining[n];
    int next[n];
    int abs[n];

    for (i=0; i<n; i++) {
        printf("Enter Capacity, Deadline,
            Period for task %d :", i+1 \n);
        scanf("%d %d %d", &C[i], &D[i],
        &T[i]);

        remaining[i] = 0;
        next[i] = 0;
        abs[i] = 0;
    }

printf("\n --EDF Scheduling -- \n");
```

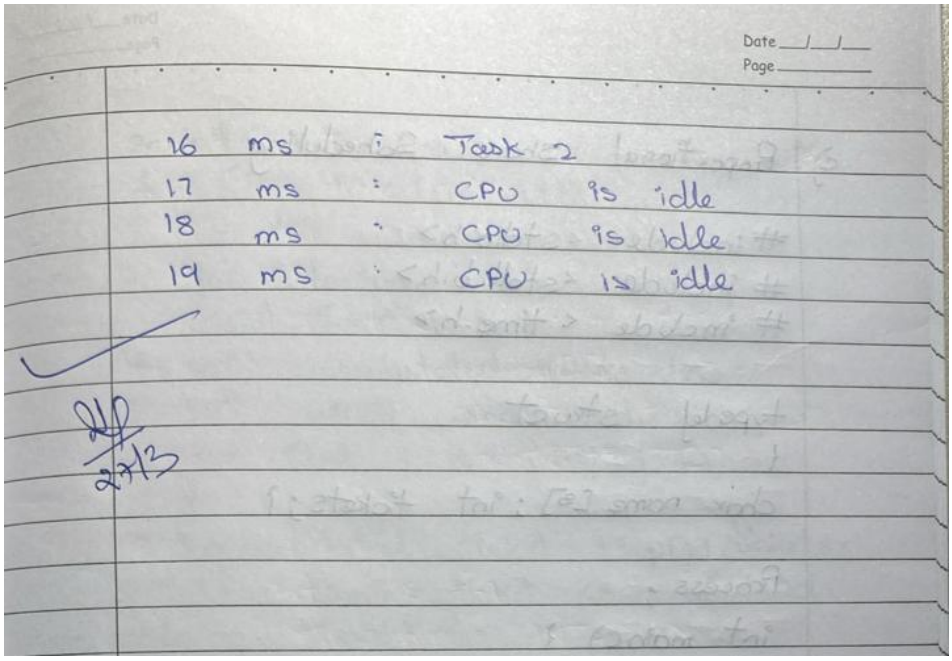


```
for (time = 0; time < hp; time ++) {
    for (i = 0; i < n; i++)
    {
        if (time == next[i]) {
            remaining[i] = c[i];
            abs[i] = time + O[i];
            next[i] += T[i];
        }
    }
    int s = -1;
    int min = INT_MAX;

    if (s == -1) {
        printf("%d ms: CPU is idle.\n",
                time);
    } else {
        printf("%d ms: Task %d is running
            .\n", time, s+1);
        remaing[s]--;
    }
}
return 0;
}
```



OUTPUT :

```
Enter no of Task : 3
Enter Capacity , DeadLine, Period :
3
7
20
2
4
5
2
8
10

-- EDF Scheduling --
0 ms : Task 2
1 ms : Task 2
2 ms : Task 1
3 ms : Task 1
4 ms : Task 1
5 ms : Task 3
6 ms : Task 3
7 ms : Task 2
8 ms : Task 2
9 ms : CPU is idle
10 ms : Task 2
11 ms : Task 2
12 ms : Task 3
13 ms : Task 3
14 ms : CPU is idle
15 ms : Task 2
```


Date
Page
16 ms : Task 2
17 ms : CPU is idle
18 ms : CPU is idle
19 ms : CPU is idle
27/3

PROPORTIONAL SHARE SCHEDULING

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
  char name[5];
  int tickets;
} Process;

int main() {
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
  int n, total_tickets = 0;
  float total_T = 0.0;

  printf("Enter the number of Processes: ");
  scanf("%d", &n);
  Process p[n];
  srand(time(NULL));

  for (int i = 0; i < n; i++) {
    printf("\nProcess %d:\n", i + 1);
    sprintf(p[i].name, "P%d", i + 1);
    printf("Tickets: ");
    scanf("%d", &p[i].tickets);
    total_tickets += p[i].tickets;
    total_T += p[i].tickets;
  }
```

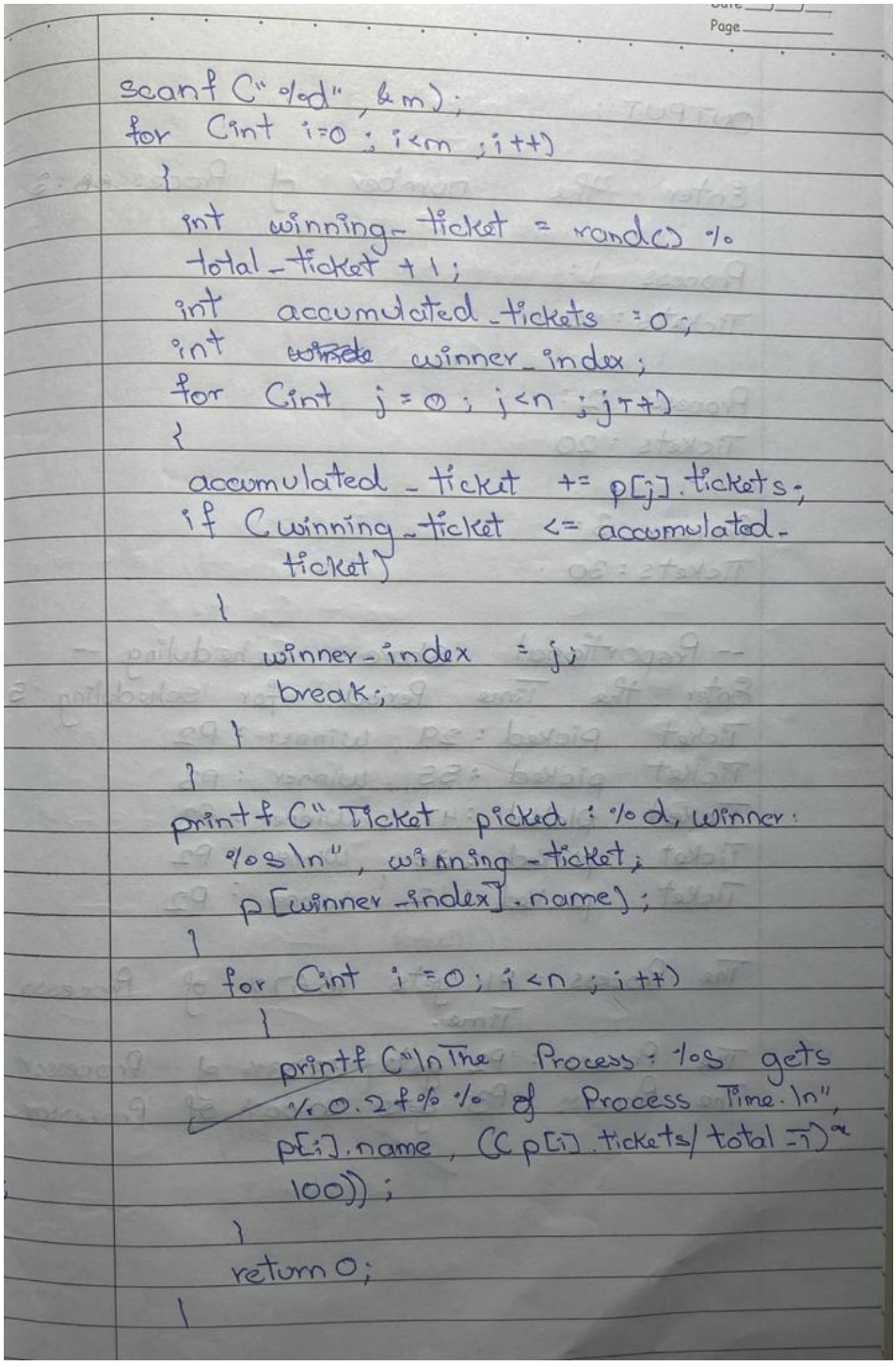
```
  printf("\n--- Proportional Share Scheduling ---\n");
  printf("Enter the Time Period for scheduling: ");
  int m;
  scanf("%d", &m);

  for (int i = 0; i < m; i++) {
    int winning_ticket = rand() % total_tickets + 1;
    int accumulated_tickets = 0;
    int winner_index;

    for (int j = 0; j < n; j++) {
      accumulated_tickets += p[j].tickets;
      if (winning_ticket <= accumulated_tickets) {
        winner_index = j;
        break;
      }
    }

    printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
          p[winner_index].name);
  }

  for (int i = 0; i < n; i++) {
    printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
          ((p[i].tickets / total_T) * 100));
  }

  return 0;
}
```

Result:

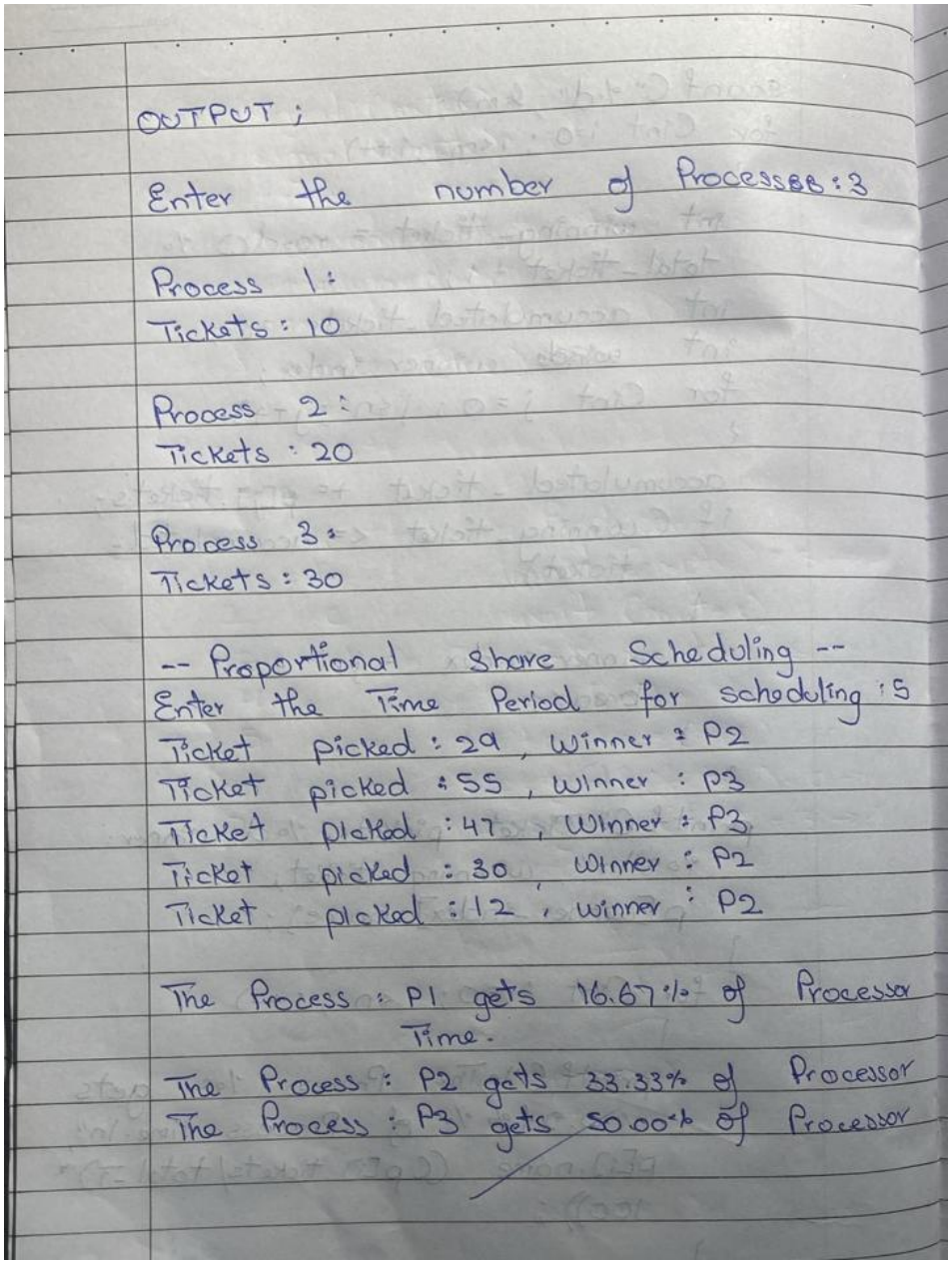
```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 57, Winner: P3
Tickets picked: 40, Winner: P3
Tickets picked: 31, Winner: P3
Tickets picked: 25, Winner: P2
Tickets picked: 9, Winner: P1

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.
```

c) Proportional share Scheduling

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct
{
char name[5]; int tickets;}

Process;
int main() {
int n, total_ticket = 0; float total_T = 0.0;
printf("Enter the number of Process :");
scanf("%d", &n);
Process p[n];
stand (time(NULL));
for(int i=0; i<n; i++)
{ printf("\n Process %d :\n", i+1);
sprintf(p[i].name, "P%d", i+1);
 printf("Tickets :");
scanf("%d", &p[i].tickets);
  total_tickets += p[i].tickets;
total_T += p[i].tickets;
}
printf("\n-- Proportional Share Scheduling --\n");
printf("Enter the Time Period for scheduling:");
int m;
```



```
scanf("%d", &m);
for (int i=0 ; i<m ; i++)
 {
  int winning_ticket = rand() %
  total_ticket + 1;
  int accumulated_tickets = 0;
  int winner_index;
  for (int j = 0 ; j<n ; j++)
  {
   accumulated_ticket += p[j].tickets;
   if (winning_ticket <= accumulated_
       ticket)
    {
     winner_index = j;
     break;
    }
  }
  printf("Ticket picked : %d, winner:
    %s\n", winning_ticket,
    p[winner_index].name);
 }
  for (int i=0; i<n ; i++)
    {
     printf("\nThe Process : %s gets
     %.0.2f%% of Process Time.\n",
     p[i].name, ((p[i].tickets/total_t)*
     100));
    }
   return 0;
}
```



OUTPUT;

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

-- Proportional Share Scheduling --
Enter the Time Period for scheduling: 5
Ticket picked: 29, Winner: P2
Ticket picked: 55, Winner: P3
Ticket picked: 47, Winner: P3
Ticket picked: 30, Winner: P2
Ticket picked: 12, winner: P2

The Process: P1 gets 16.67% of Processor Time.
The Process: P2 gets 33.33% of Processor
The Process: P3 gets 50.00% of Processor

Program -4

Question:

Write a C program to simulate: (Any one)
a) Producer-Consumer problem using semaphores.
b) Dining-Philosopher's problem

PRODUCER – CONSUMER

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int top = -1;
int item = 0;

sem_t empty, full, mutex;


void produce() {
   sem_wait(&empty);
   sem_wait(&mutex);

   item++;
   top++;
   buffer[top] = item;

   printf("Producer has produced: Item %d\n", item);
```

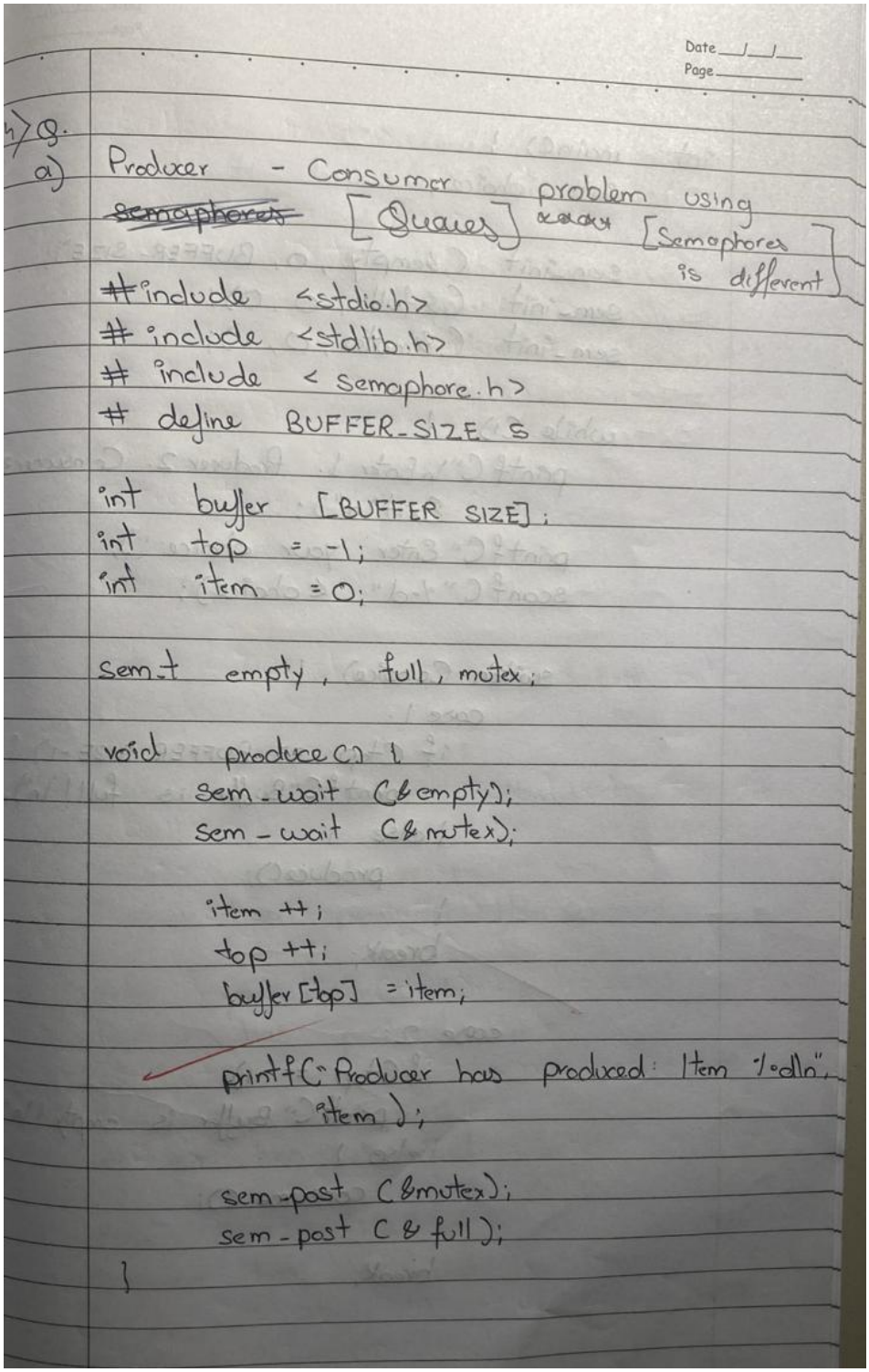
```
    sem_post(&mutex);
    sem_post(&full);
}

void consume() {
    sem_wait(&full);
    sem_wait(&mutex);

    int consumed = buffer[top];
    printf("Consumer has consumed: Item %d\n", consumed);

    top--;

    sem_post(&mutex);
    sem_post(&empty);
}

int main() {
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
    int choice;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    while (1) {
        printf("\nEnter 1. Producer 2. Consumer 3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
```

```
            if (top == BUFFER_SIZE - 1) {
                printf("Buffer is full!\n");
            } else {
                produce();
            }
            break;

        case 2:
            if (top == -1) {
                printf("Buffer is empty!\n");
            } else {
                consume();
            }
            break;

        case 3:
            printf("Exiting...\n");
            exit(0);

        default:
            printf("Invalid choice!\n");
      }
   }

   return 0;
}
```

Result:

```
Name: PRATHEEK P REDDY
USN:1WA24CS215


Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 1
Producer has produced: Item 1

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 1
Producer has produced: Item 2

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 1
Producer has produced: Item 3

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 2
Consumer has consumed: Item 3

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 2
Consumer has consumed: Item 2

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 2
Consumer has consumed: Item 1
```

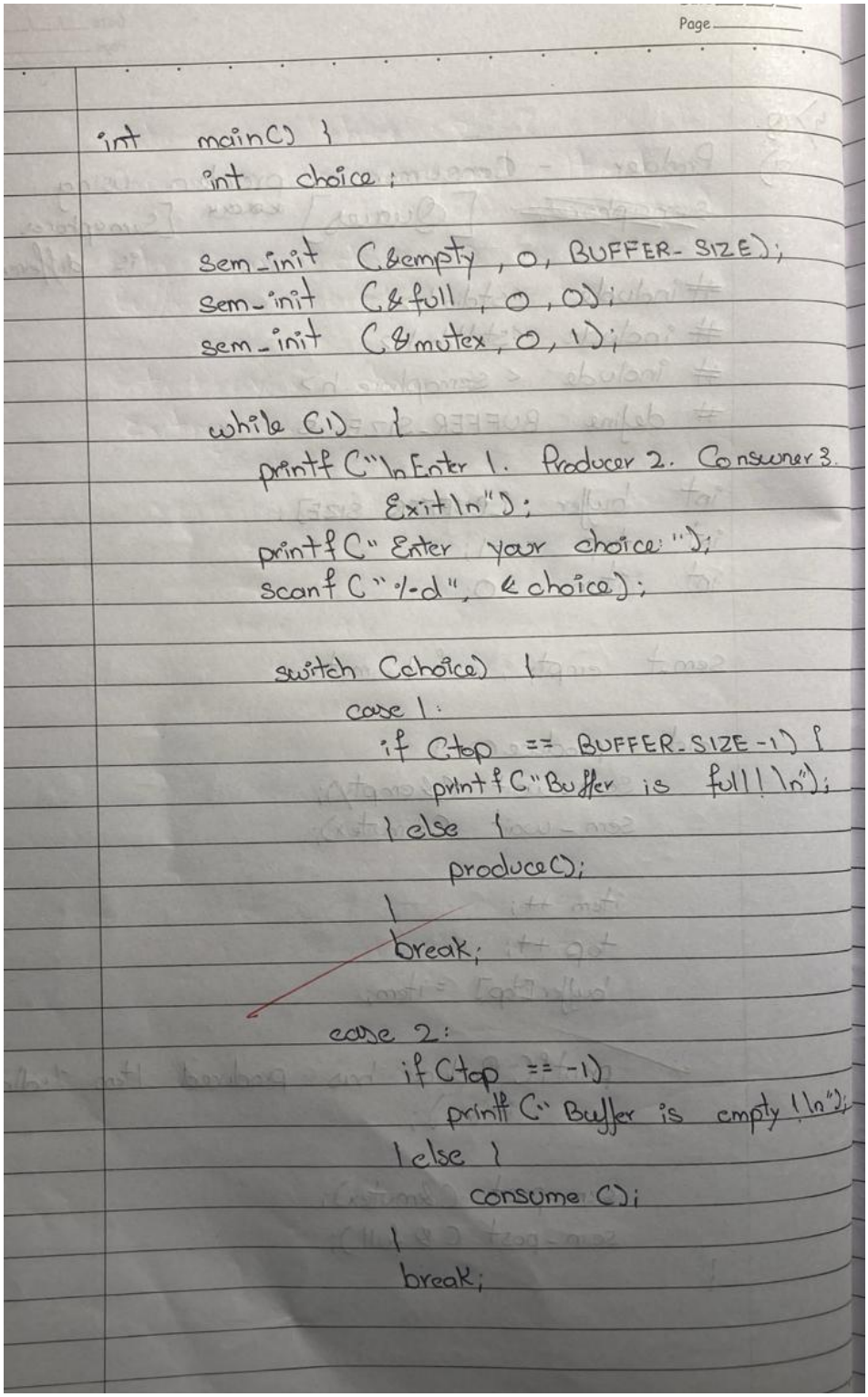
```
Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 3
Exiting...


...Program finished with exit code 0
Press ENTER to exit console.
```



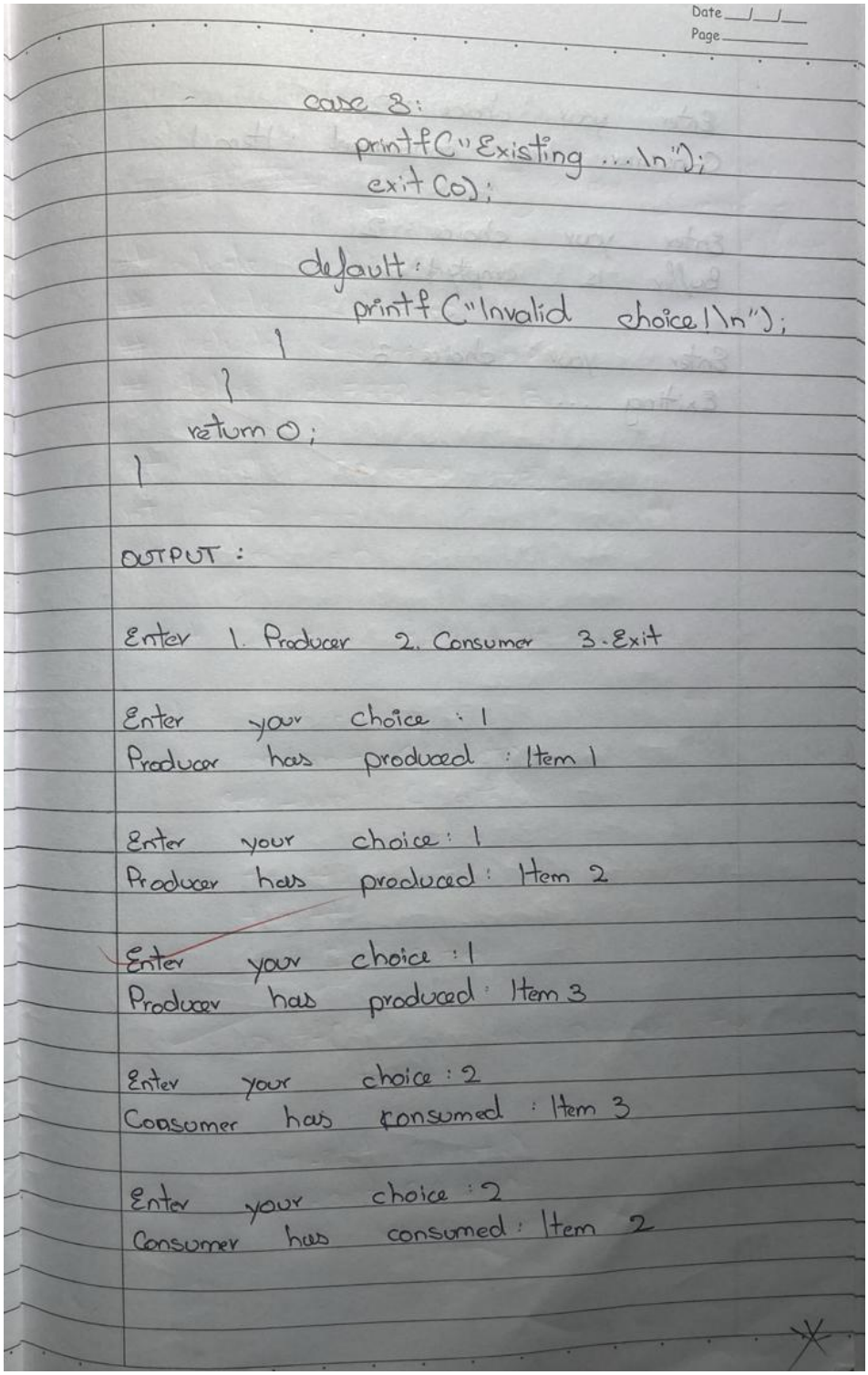
Date __/__/__
Page _____

4) Q.
a) Producer - Consumer problem using ~~Semaphores~~ [Queues] already [Semaphores is different]

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#define BUFFER_SIZE 5

int buffer [BUFFER SIZE];
int top = -1;
int item = 0;

sem_t empty, full, mutex;

void produce() {
    sem_wait (&empty);
    sem_wait (&mutex);

    item ++;
    top ++;
    buffer[top] = item;

    printf("Producer has produced: Item %d\n",
        item);

    sem_post (&mutex);
    sem_post (&full);
}
```



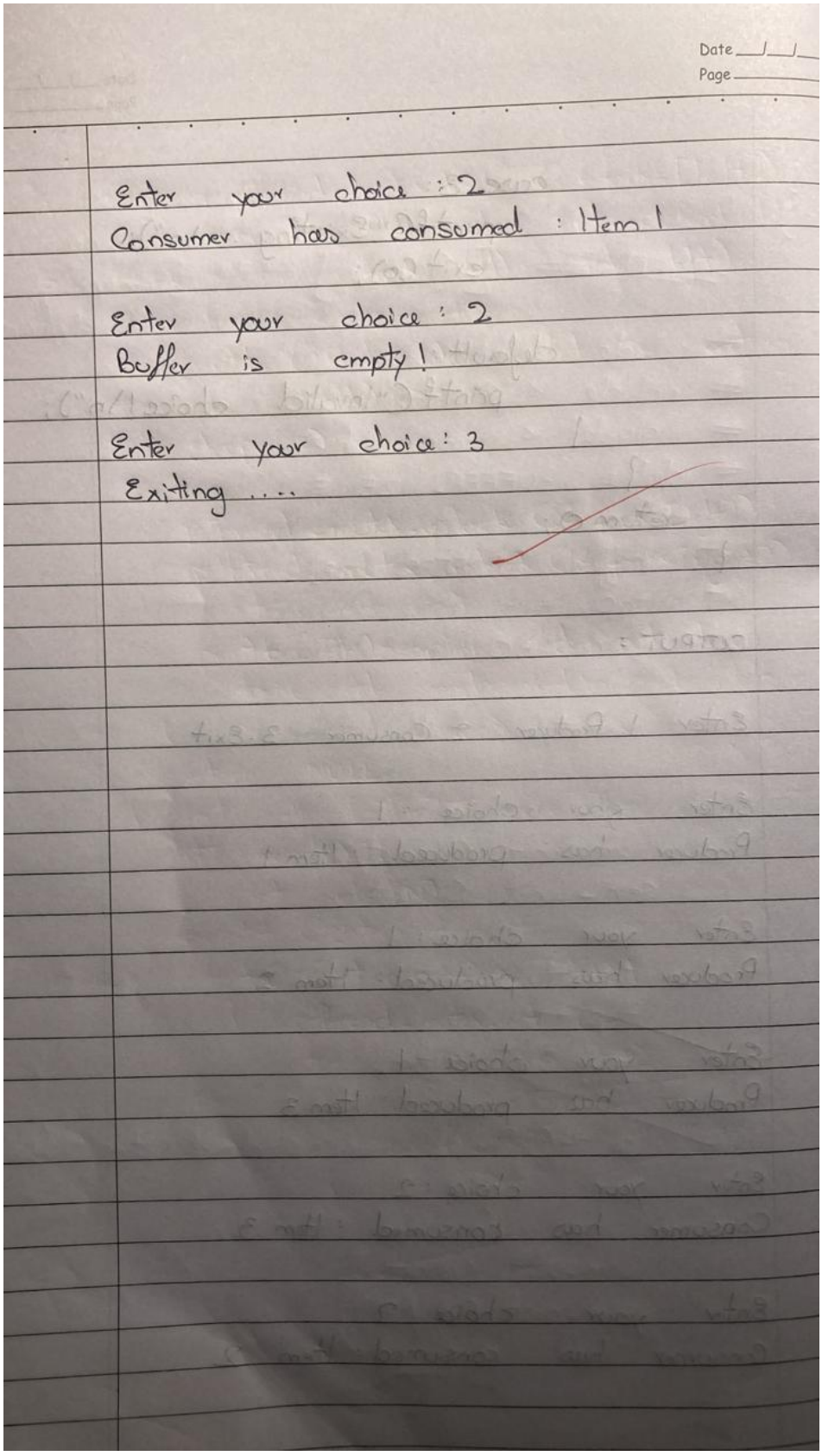
```
int main() {
    int choice;

    sem_init (&empty, 0, BUFFER_SIZE);
    sem_init (&full, 0, 0);
    sem_init (&mutex, 0, 1);

    while (1) {
        printf ("\n Enter 1. Producer 2. Consumer 3.
                Exit\n");
        printf (" Enter your choice: ");
        scanf ("%d", &choice);

        switch (choice) {
            case 1:
                if (top == BUFFER_SIZE - 1) {
                    printf ("Buffer is full!! \n");
                } else {
                    produce();
                }
                break;

            case 2:
                if (top == -1)
                    printf (" Buffer is empty !!\n");
                else {
                    consume ();
                }
                break;
```



```
        case 3:
            printf("Existing ...\n");
            exit(0);

        default:
            printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

OUTPUT :

```
Enter 1. Producer 2. Consumer 3. Exit

Enter your choice : 1
Producer has produced : Item 1

Enter your choice : 1
Producer has produced : Item 2

Enter your choice : 1
Producer has produced : Item 3

Enter your choice : 2
Coasumer has consumed : Item 3

Enter your choice : 2
Consumer has consumed : Item 2
```


Date
Page
Enter your choice : 2
Consumer has consumed : Item 1
Enter your choice : 2
Buffer is empty!
Enter your choice : 3
Exiting ....

DINING – PHILOSOPHER'S

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];


void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;

    for (int i = 0; i < 1; i++) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked left fork %d\n", id, left);

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked right fork %d\n", id, right);
        } else {
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked right fork %d\n", id, right);

            pthread_mutex_lock(&forks[left]);   // FIXED TYPO HERE
```

```
            printf("Philosopher %d picked left fork %d\n", id, left);
        }

        printf("Philosopher %d is eating.\n", id);
        sleep(2);

        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);

        printf("Philosopher %d put down forks %d and %d\n", id, left, right);
    }

    return NULL;
}

int main() {
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
    int ids[N];


    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }


    for (int i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
```

```
    }

    for (int i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```

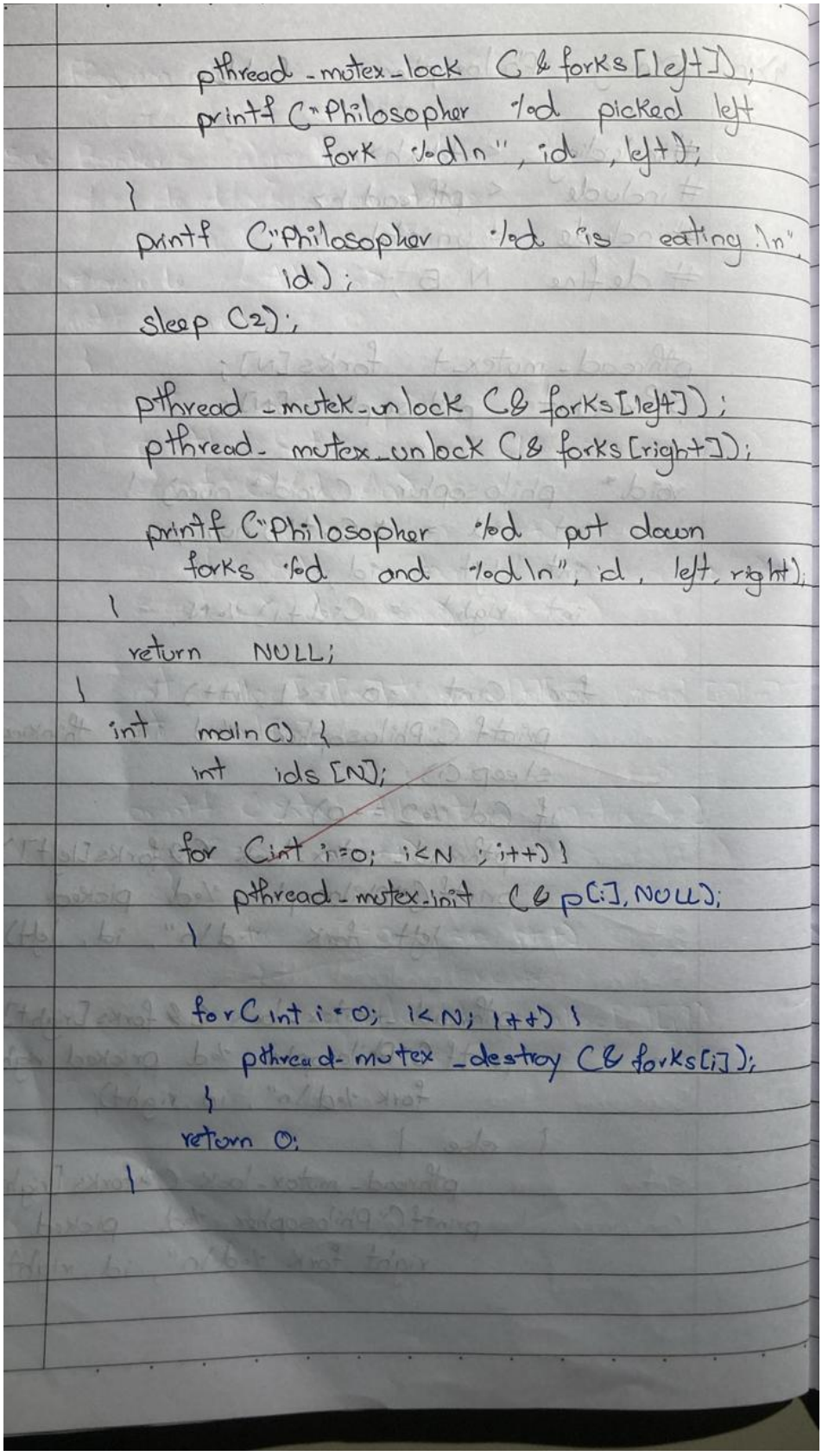
Result:

```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 2 picked left fork 2
Philosopher 2 picked right fork 3
Philosopher 2 is eating.
Philosopher 0 picked left fork 0
Philosopher 0 picked right fork 1
Philosopher 0 is eating.
Philosopher 3 picked right fork 4
Philosopher 0 put down forks 0 and 1
Philosopher 1 picked right fork 2
Philosopher 1 picked left fork 1
Philosopher 1 is eating.
Philosopher 2 put down forks 2 and 3
Philosopher 3 picked left fork 3
Philosopher 3 is eating.
Philosopher 1 put down forks 1 and 2
Philosopher 3 put down forks 3 and 4
Philosopher 4 picked left fork 4
Philosopher 4 picked right fork 0
Philosopher 4 is eating.
Philosopher 4 put down forks 4 and 0


...Program finished with exit code 0
Press ENTER to exit console.
```

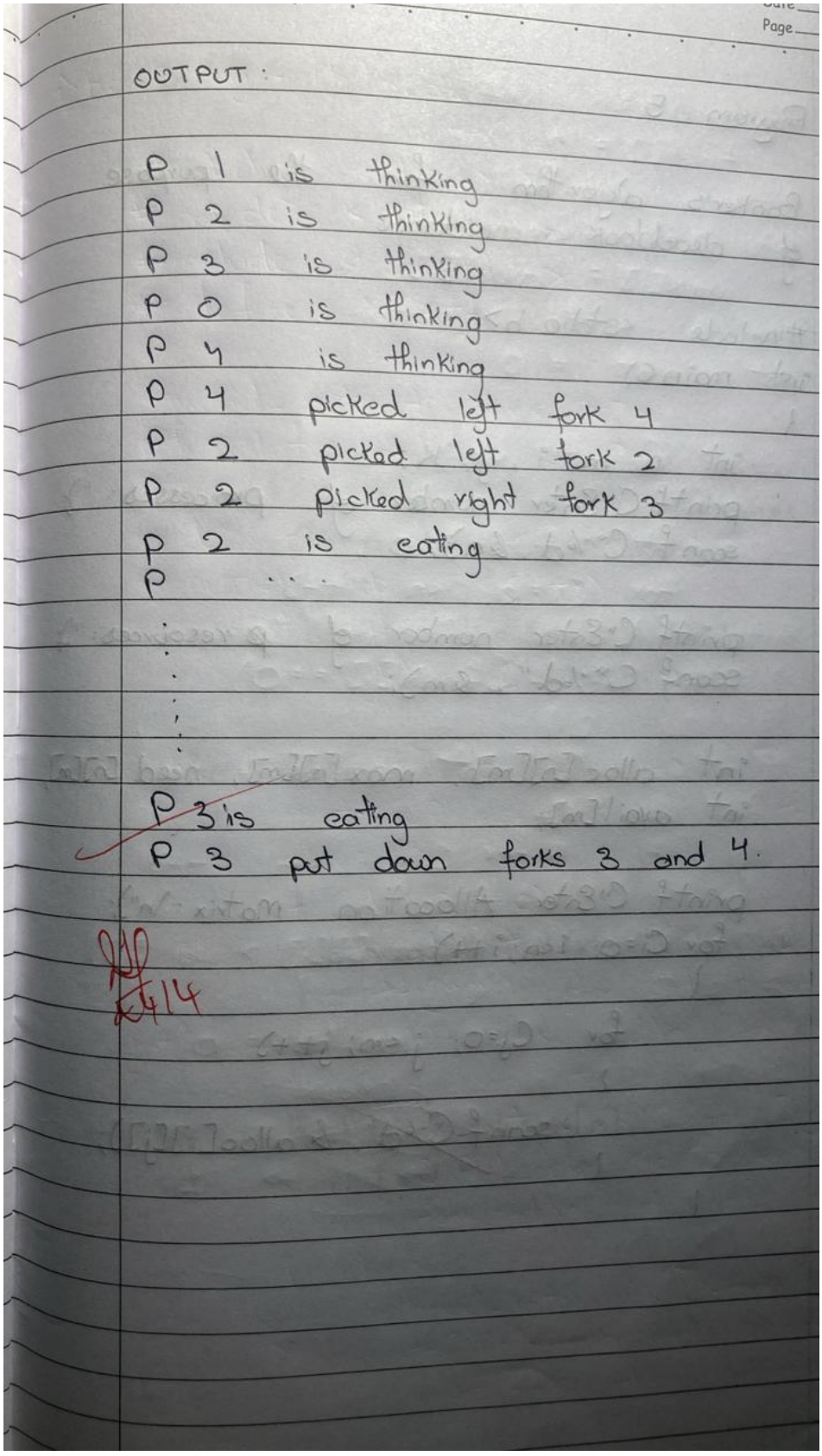
Date / /
Page

4) b. Dining - Philosopher's Problem

```
#including <stdio.h>
#include < pthread.h>
#include < unistd.h >
# define N 5;

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher (void* num) {
    int id = *(int*) num;
    int left = id;
    int right = (id+1) % N;

    for (int i=0; i<1; i++) {
        printf("Philosopher %d is thinking.\n",
        sleep(1);
        if (id %2 == 0) {
            pthread_mutex_lock (&forks[left]);
            printf("Philosopher %d picked
                left fork %d\n", id, left);

            pthread_mutex_lock (&forks[right]);
            printf("Philosopher %d pricked right
                fork %d\n", id, right);
        } else {
            pthread_mutex_lock (&forks[right]);
            printf("Philosopher %d picked
                right fork %d\n", id, right);
```



```
        pthread_mutex_lock (&forks[left]);
        printf("Philosopher %d picked left
                fork %d\n", id, left);
    }
    printf("Philosopher %d is eating.\n",
            id);
    sleep(2);

    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);

    printf("Philosopher %d put down
      forks %d and %d\n", id, left, right);
}
    return NULL;
}
int main() {
    int ids[N];

    for(int i=0; i<N; i++) {
        pthread_mutex_init(&p[i], NULL);
    }

    for(int i=0; i<N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }
    return 0;
}
```



OUTPUT :

P 1 is thinking
P 2 is thinking
P 3 is thinking
P 0 is thinking
P 4 is thinking
P 4 picked left fork 4
P 2 picked left fork 2
P 2 picked right fork 3
P 2 is eating
P ...

P 3 is eating
P 3 put down forks 3 and 4.

4/4

Program -5

Question:

Write a C program to simulate: (Any one)
a) Bankers' algorithm for the purpose of deadlock avoidance.
b) Deadlock Detection

BANKER'S ALGORITHM

CODE:

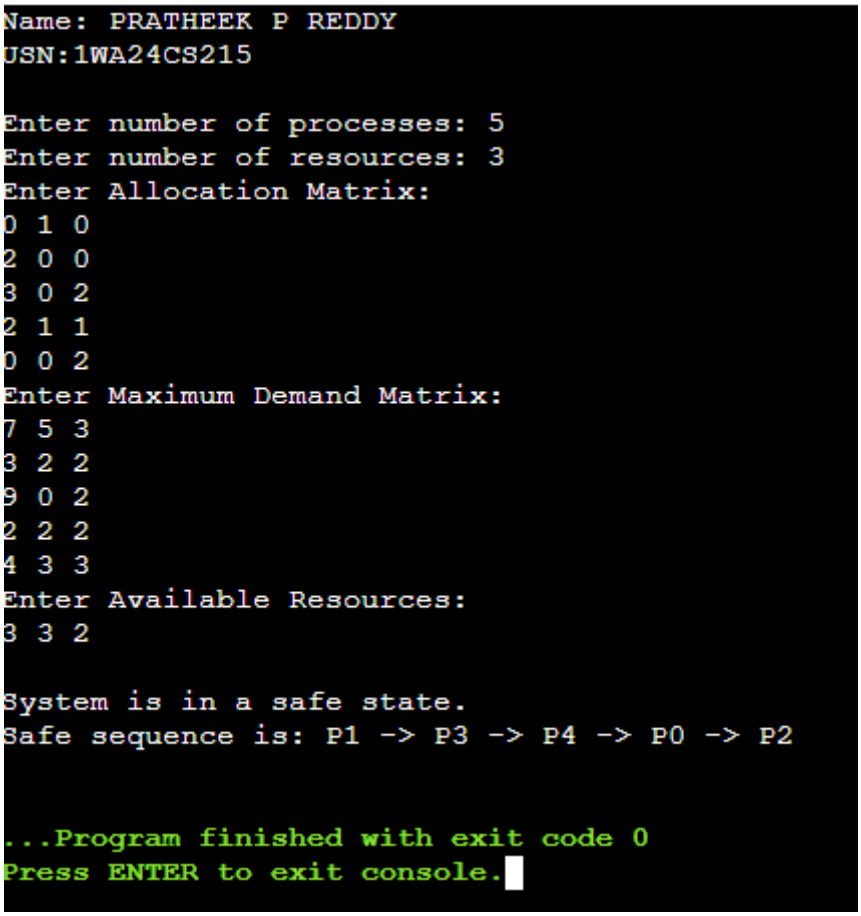
```
#include <stdio.h>

int main()
{
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
   int n, m, i, j, k;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);

   int alloc[n][m], max[n][m], need[n][m];
   int avail[m];

   printf("Enter Allocation Matrix:\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         scanf("%d", &alloc[i][j]);
      }
   }
```

```
printf("Enter Maximum Demand Matrix:\n");
for(i = 0; i < n; i++)
{
   for(j = 0; j < m; j++)
   {
      scanf("%d", &max[i][j]);
   }
}

printf("Enter Available Resources:\n");
for(i = 0; i < m; i++)
{
   scanf("%d", &avail[i]);
}

// Calculate Need Matrix
for(i = 0; i < n; i++)
{
   for(j = 0; j < m; j++)
   {
      need[i][j] = max[i][j] - alloc[i][j];
   }
}

int finish[n], safeSeq[n];
int work[m];

for(i = 0; i < n; i++)
   finish[i] = 0;

for(i = 0; i < m; i++)
```

```
    work[i] = avail[i];

int count = 0;

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;

            for(j = 0; j < m; j++)
            {
                if(need[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }

            if(possible)
            {
                for(k = 0; k < m; k++)
                {
                    work[k] += alloc[i][k];
                }

                safeSeq[count] = i;
                count++;
```

```
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
    {
        printf("System is not in a safe state.\n");
        return 0;
    }
}

printf("\nSystem is in a safe state.\n");
printf("Safe sequence is: ");

for(i = 0; i < n; i++)
{
    printf("P%d", safeSeq[i]);

    if(i != n - 1)
        printf(" -> ");
}

printf("\n");

return 0;
}
```

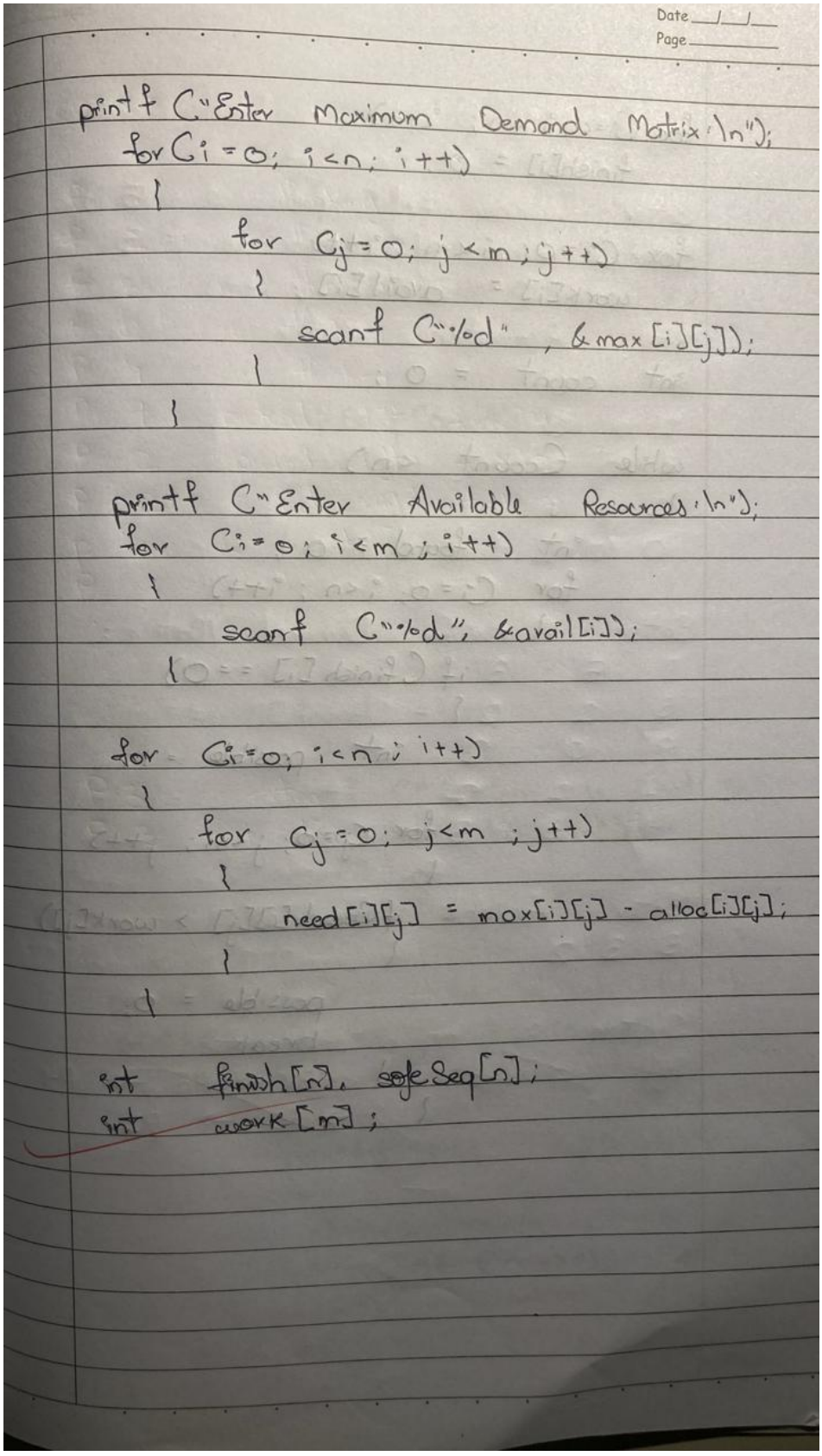
Result:


Name: PRATHEEK P REDDY
USN:1WA24CS215
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2
System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
...Program finished with exit code 0
Press ENTER to exit console.

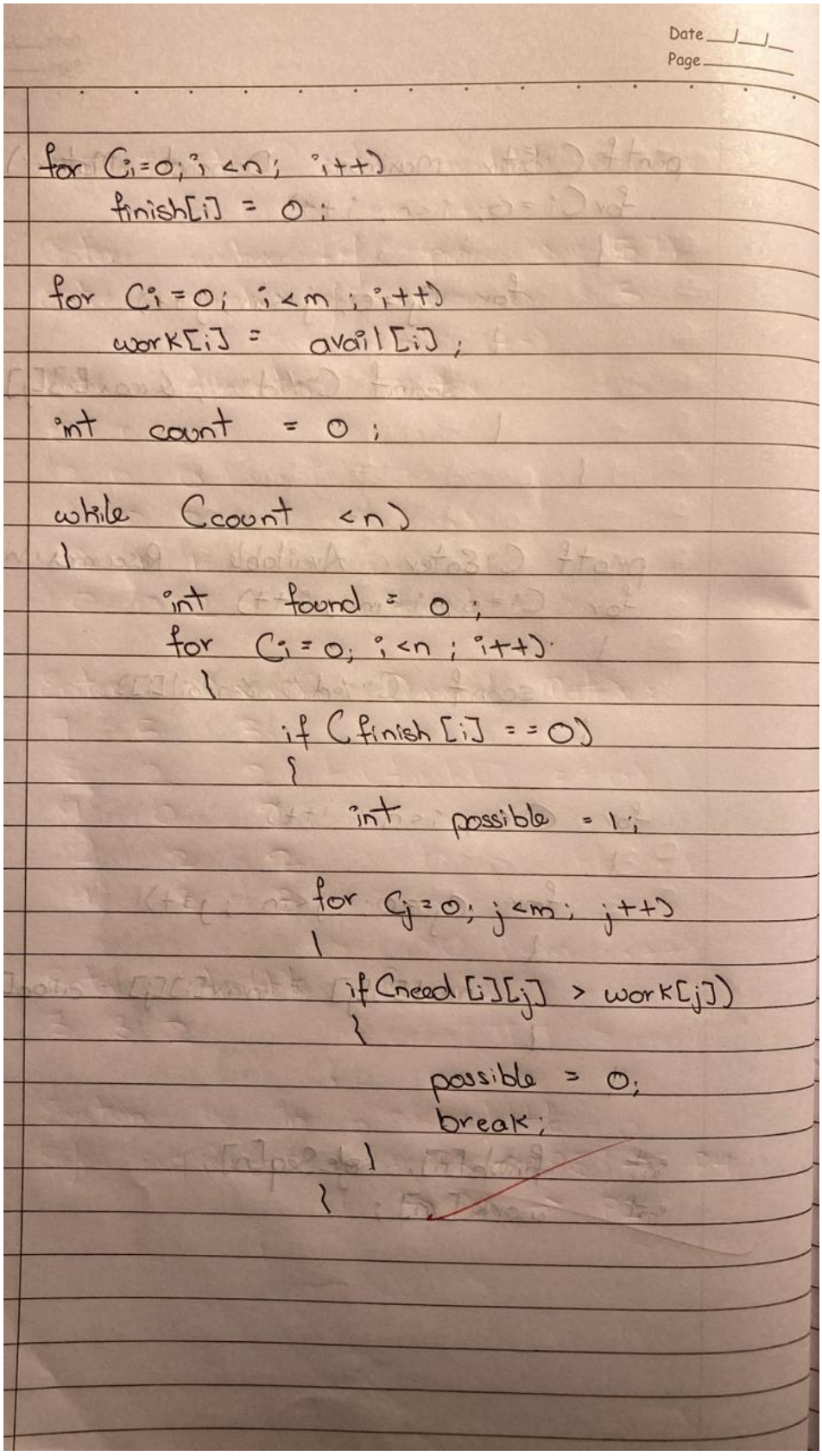
Program 5

5a. Banker's algorithm for the purpose of deadlock avoidance

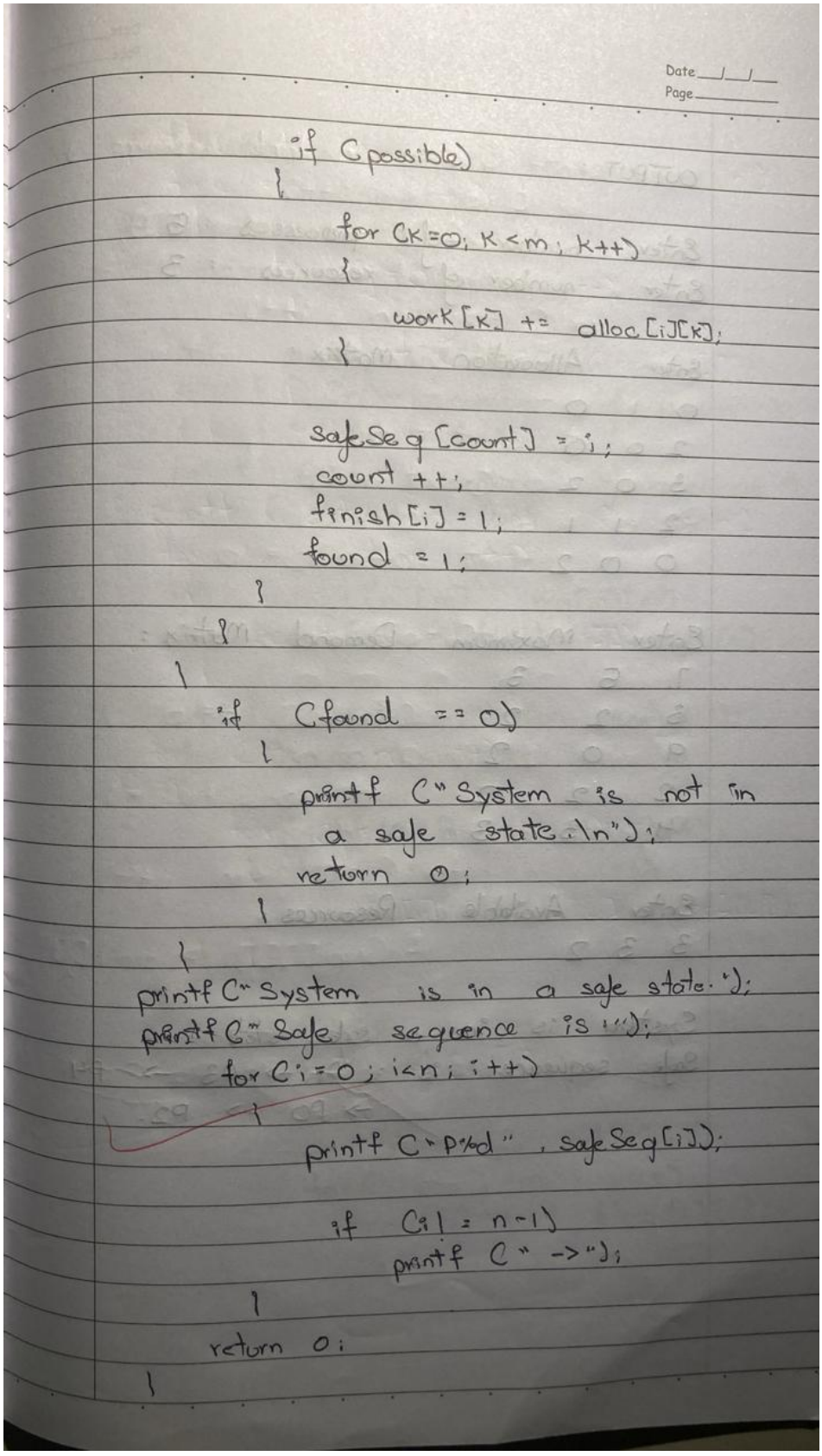
```
#include <stdio.h>
int main()
{
    int n, m, i, j, k;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("Enter Allocation Matrix: \n");
    for(i=0; i<n; i++)
    {
        for(j=0; j<m; j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }
```



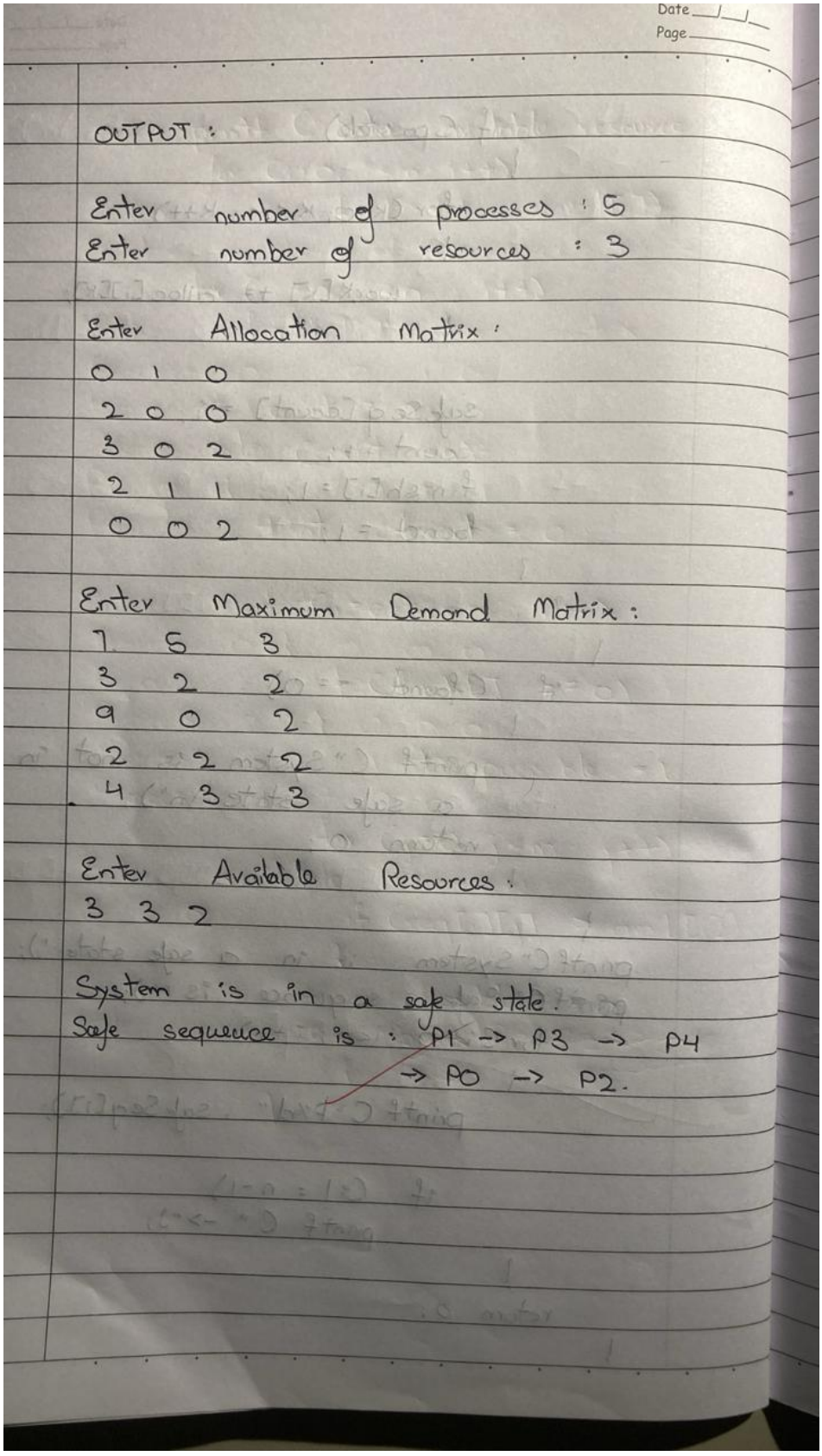
```
printf("Enter Maximum Demand Matrix:\n");
for(i=0; i<n; i++)
{
    for(j=0; j<m; j++)
    {
        scanf("%d", &max[i][j]);
    }
}

printf("Enter Available Resources:\n");
for(i=0; i<m; i++)
{
    scanf("%d", &avail[i]);
}

for(i=0; i<n; i++)
{
    for(j=0; j<m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int finish[n], safeSeq[n];
int work[m];
```



```
for (i=0; i<n; i++)
    finish[i] = 0;

for (i=0; i<m; i++)
    work[i] = avail[i];

int count = 0;

while (count < n)
{
    int found = 0;
    for (i=0; i<n; i++)
    {
        if (finish[i] == 0)
        {
            int possible = 1;

            for (j=0; j<m; j++)
            {
                if (need[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }
```



```
        if (possible)
        {
            for (k=0; k<m; k++)
            {
                work[k] += alloc[i][k];
            }

            safeSeq[count] = i;
            count++;
            finish[i] = 1;
            found = 1;
        }
    }
}
if (found == 0)
{
    printf("System is not in
     a safe state.\n");
    return 0;
}
}
printf("System is in a safe state.");
printf("Safe sequence is:");
    for(i=0; i<n; i++)
    {
        printf("%d", safeSeq[i]);

        if (i != n-1)
            printf(" ->");
    }
return 0;
}
```



OUTPUT :

Enter number of processes : 5
Enter number of resources : 3

Enter Allocation Matrix :

0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Demond Matrix :

7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources :

3 3 2

System is in a safe state.
Safe sequence is : P1 -> P3 -> P4
-> P0 -> P2.

DEADLOCK DETECTION

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");
   int n, m, i, j, k;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);

   int alloc[n][m], req[n][m];
   int avail[m], finish[n], safe[n];

   printf("Enter allocation matrix:\n");
   for(i = 0; i < n; i++)
   {
      printf("Process %d: ", i);
      for(j = 0; j < m; j++)
         scanf("%d", &alloc[i][j]);
   }

   printf("Enter request matrix:\n");
   for(i = 0; i < n; i++)
   {
      printf("Process %d: ", i);
      for(j = 0; j < m; j++)
```

```
        scanf("%d", &req[i][j]);
}

printf("Enter available resources: ");
for(i = 0; i < m; i++)
   scanf("%d", &avail[i]);

for(i = 0; i < n; i++)
   finish[i] = 0;

int count = 0;

while(count < n)
{
   int found = 0;

   for(i = 0; i < n; i++)
   {
      if(finish[i] == 0)
      {
         int possible = 1;

         for(j = 0; j < m; j++)
         {
            if(req[i][j] > avail[j])
            {
               possible = 0;
               break;
            }
         }

         if(possible)
```

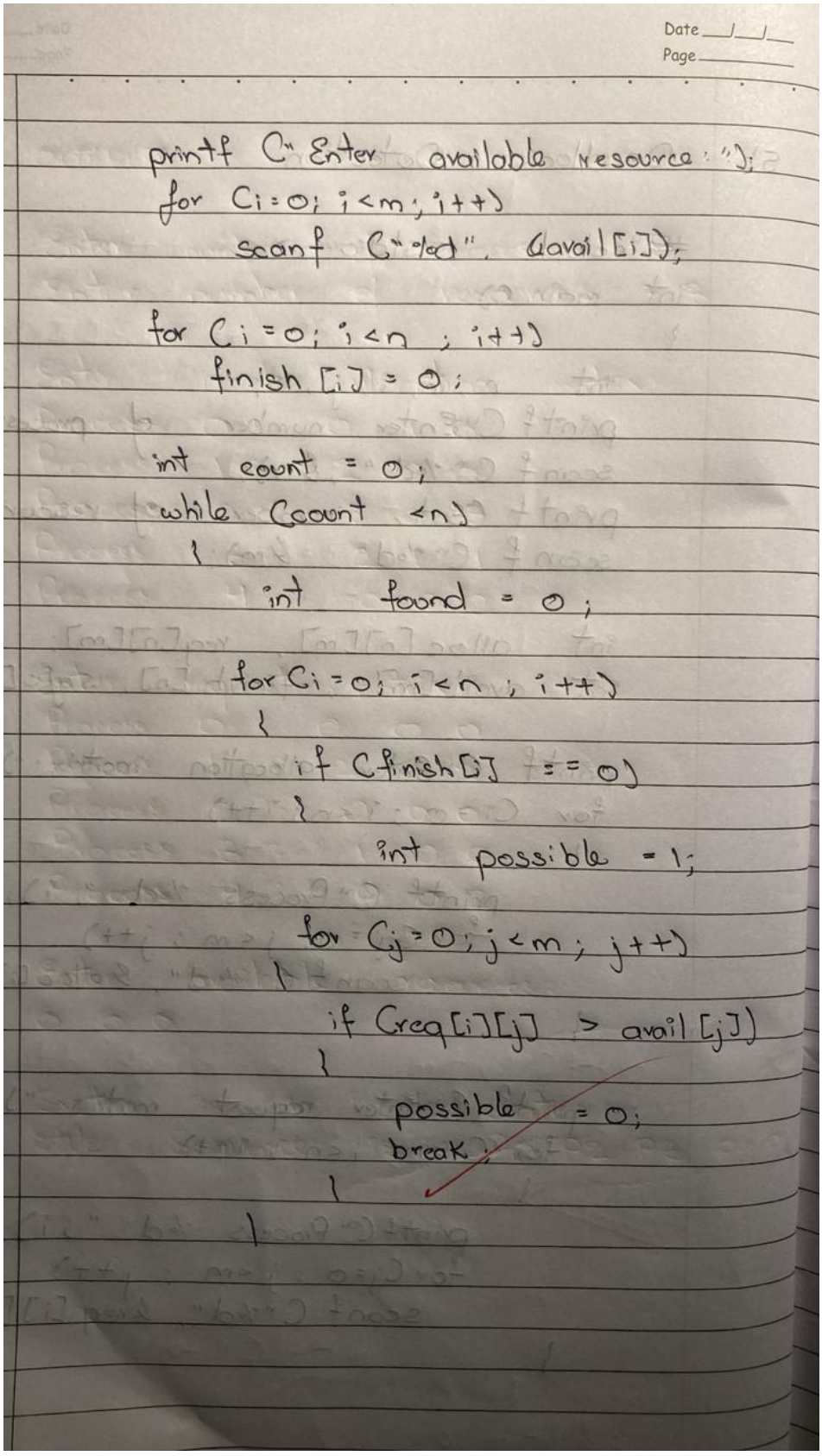
```
            {
                for(k = 0; k < m; k++)
                    avail[k] += alloc[i][k];

                safe[count] = i;
                count++;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
        break;
}

if(count == n)
{
    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

    for(i = 0; i < n; i++)
        printf("P%d ", safe[i]);
}
else
{
    printf("\nDeadlock detected.\n");
}

return 0;
}
```

Result:

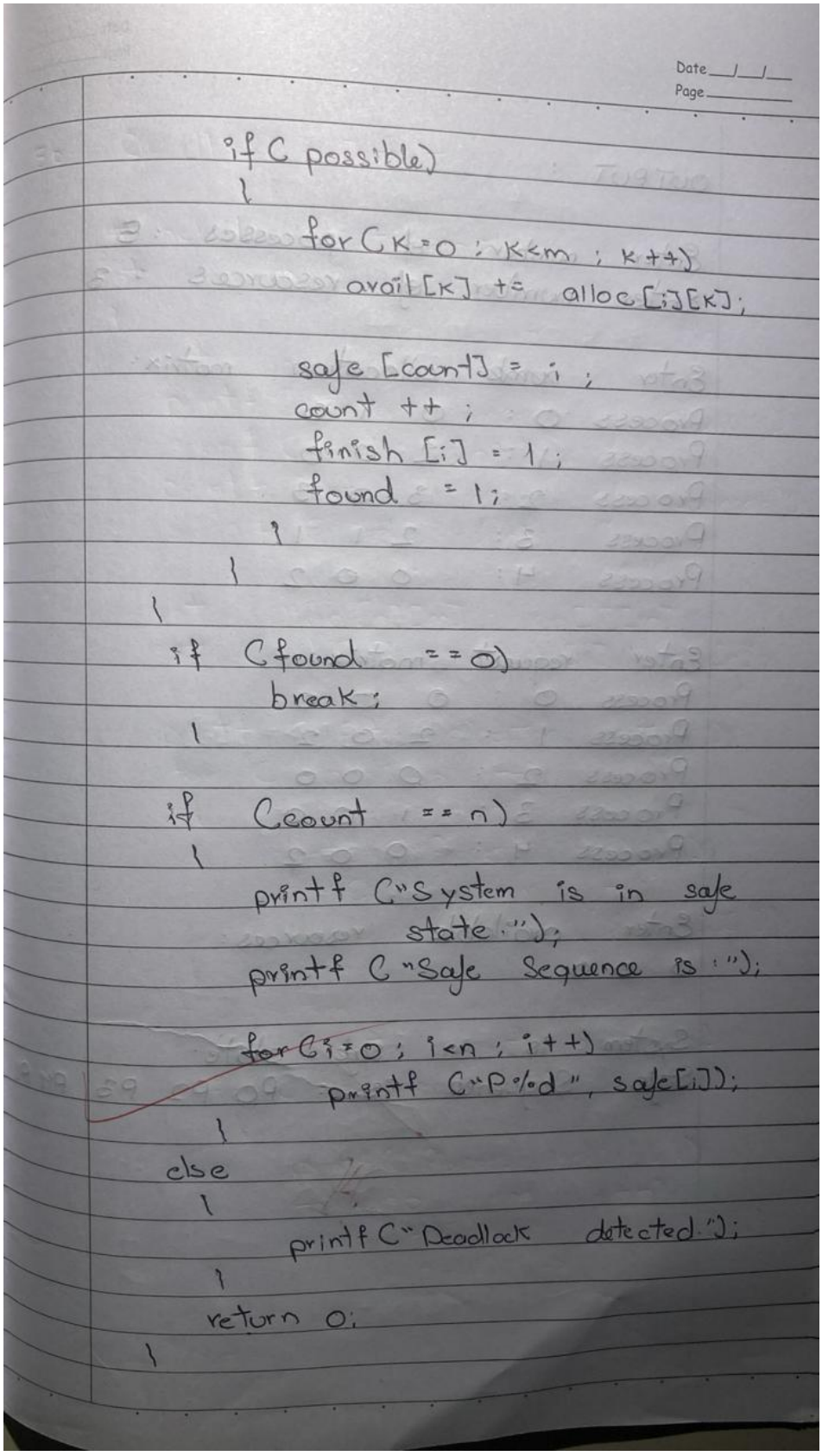
```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter number of processes: 5
Enter number of resources: 3
Enter allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter available resources: 0 0 0

System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1

...Program finished with exit code 0
Press ENTER to exit console.
```

5b. Deadlock Detection

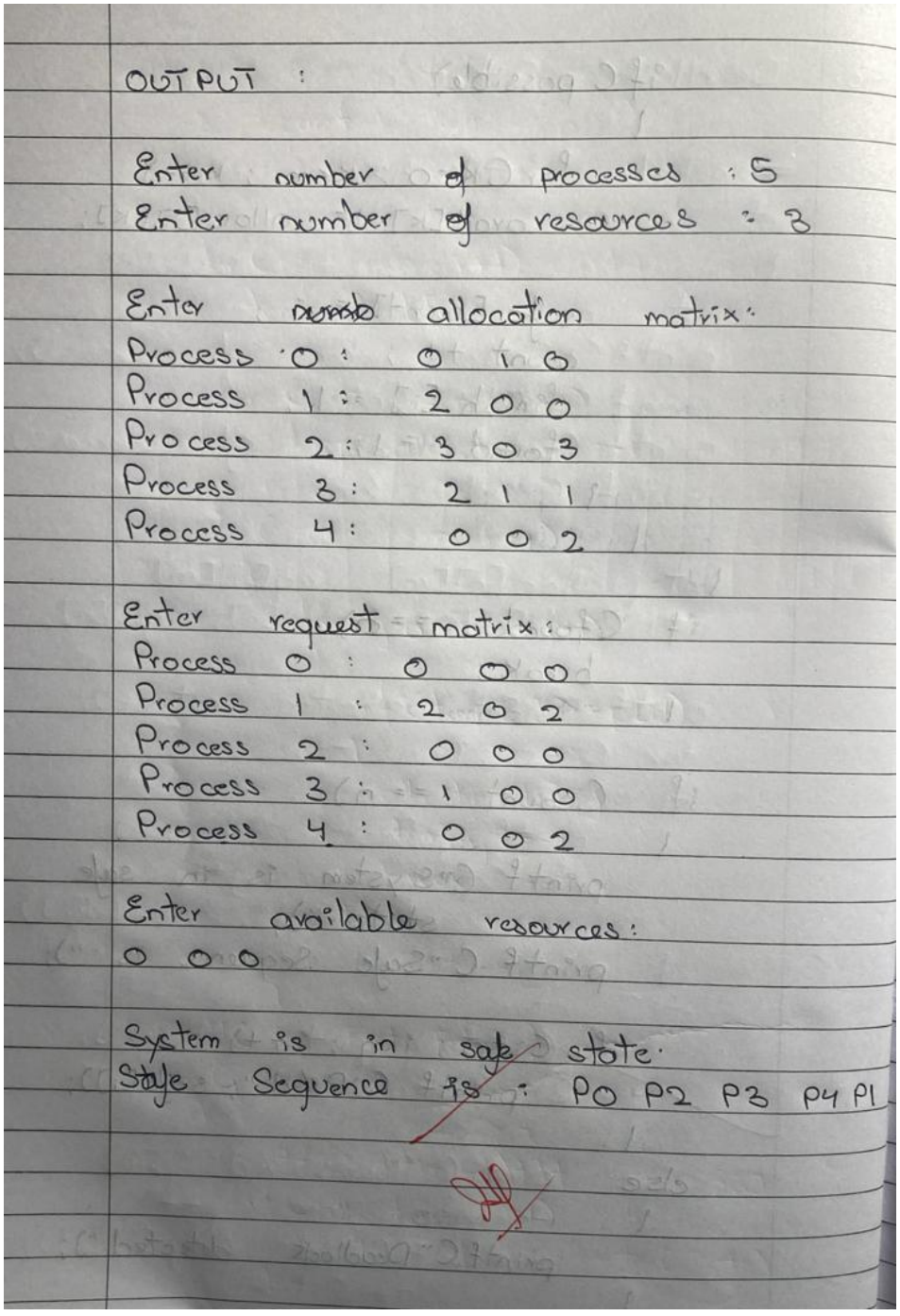
```
#include <stdio.h>
int main()
{
    int n, m, i, j, k;
    printf("Enter number of process: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], req[n][m];
    int avail[m], finish[n], safe[n];

    printf("Enter allocation matrix: ");
    for(i=0; i<n; i++)
    {
        printf("Process %d: ", i);
        for(j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);
    }

    printf("Enter request matrix: ");
    for(i=0; i<n; i++)
    {
        printf("Process %d: ", i);
        for(j=0; j<m; j++)
            scanf("%d", &req[i][j]);
    }
```



```
printf("Enter available resource: ");
for(i=0; i<m; i++)
    scanf("%d", &avail[i]);

for(i=0; i<n ; i++)
    finish[i] = 0;

int count = 0;
while(count <n)
{
    int found = 0;

    for(i=0; i<n ; i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;

            for(j=0; j<m; j++)
            {
                if(req[i][j] > avail[j])
                {
                    possible = 0;
                    break;
                }
            }
```



```
if (possible)
{
    for(k=0; k<m; k++)
        avail[k] += alloc[i][k];

    safe[count] = i;
    count++;
    finish[i] = 1;
    found = 1;
  }
 }
}
if (found == 0)
    break;
}

if (count == n)
{
    printf("System is in safe
            state.");
    printf("Safe Sequence is:");

    for(i=0; i<n; i++)
        printf("P%d", safe[i]);
  }
else
{
    printf("Deadlock detected.");
}
return 0;
}
```



OUTPUT :

Enter number of processes : 5
Enter number of resources : 3

Enter allocation matrix:
Process 0 : 0 1 0
Process 1 : 2 0 0
Process 2 : 3 0 3
Process 3 : 2 1 1
Process 4 : 0 0 2

Enter request matrix:
Process 0 : 0 0 0
Process 1 : 2 0 2
Process 2 : 0 0 0
Process 3 : 1 0 0
Process 4 : 0 0 2

Enter available resources:
0 0 0

System is in safe state.
Safe Sequence is : P0 P2 P3 P4 P1

Program -6

Question:

Write a C program to simulate the following contiguous memory allocation techniques. (Any one)
a) Worst-fit
b) Best-fit
c) First-fit

CODE:

```
#include <stdio.h>

void firstFit(int blocks[], int m, int processes[], int n) {
  int alloc[n], used[m];
  for (int i = 0; i < m; i++)
    used[i] = 0;
  for (int i = 0; i < n; i++)
    alloc[i] = -1;

  for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
      if (!used[j] && blocks[j] >= processes[i]) {
        alloc[i] = j;
        used[j] = 1;
        break;
      }

  printf("\n--- First Fit ---\n");
  printf("Process No.\tProcess Size\tBlock No.\n");
  for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processes[i]);
    if (alloc[i] != -1)
      printf("%d\n", alloc[i] + 1);
    else
```

```
      printf("Not Allocated\n");
    }
  }

void bestFit(int blocks[], int m, int processes[], int n) {
  int alloc[n], used[m];
  for (int i = 0; i < m; i++)
    used[i] = 0;
  for (int i = 0; i < n; i++)
    alloc[i] = -1;

  for (int i = 0; i < n; i++) {
    int best = -1;
    for (int j = 0; j < m; j++)
      if (!used[j] && blocks[j] >= processes[i])
        if (best == -1 || blocks[j] < blocks[best])
          best = j;
    if (best != -1) {
      alloc[i] = best;
      used[best] = 1;
    }
  }

  printf("\n--- Best Fit ---\n");
  printf("Process No.\tProcess Size\tBlock No.\n");
  for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processes[i]);
    if (alloc[i] != -1)
      printf("%d\n", alloc[i] + 1);
    else
      printf("Not Allocated\n");
  }
```

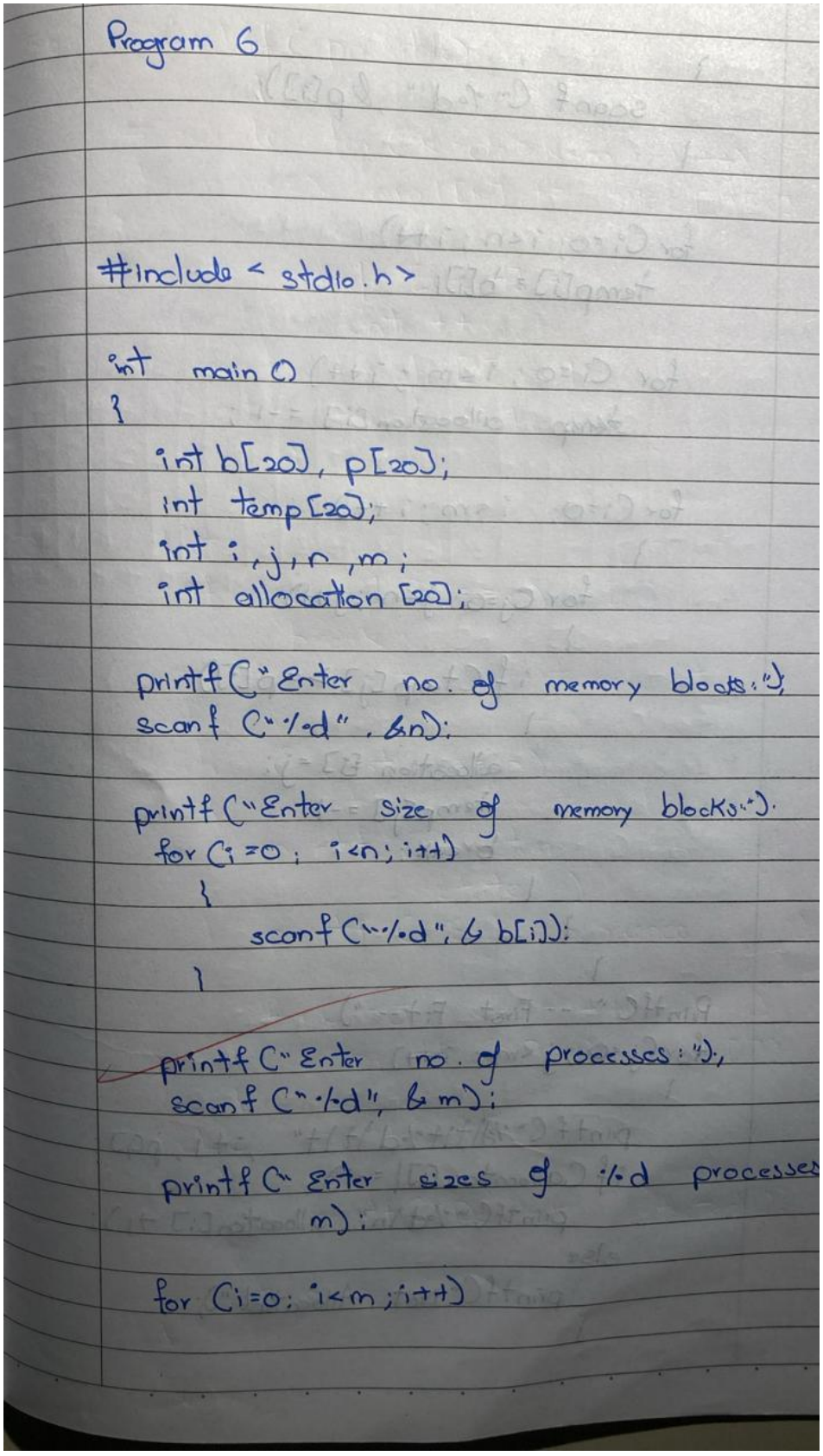
```
}

void worstFit(int blocks[], int m, int processes[], int n) {
  int alloc[n], used[m];
  for (int i = 0; i < m; i++)
    used[i] = 0;
  for (int i = 0; i < n; i++)
    alloc[i] = -1;

  for (int i = 0; i < n; i++) {
    int worst = -1;
    for (int j = 0; j < m; j++)
      if (!used[j] && blocks[j] >= processes[i])
        if (worst == -1 || blocks[j] > blocks[worst])
          worst = j;
    if (worst != -1) {
      alloc[i] = worst;
      used[worst] = 1;
    }
  }

  printf("\n--- Worst Fit ---\n");
  printf("Process No.\tProcess Size\tBlock No.\n");
  for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processes[i]);
    if (alloc[i] != -1)
      printf("%d\n", alloc[i] + 1);
    else
      printf("Not Allocated\n");
  }
}
```

```
int main() {

  printf("Name: PRATHEEK P REDDY\nUSN:1WA24CS215\n\n");

  int m, n;

  printf("Enter number of memory blocks: ");
  scanf("%d", &m);
  int blocks[m];
  printf("Enter sizes of %d memory blocks:\n", m);
  for (int i = 0; i < m; i++)
    scanf("%d", &blocks[i]);

  printf("Enter number of processes: ");
  scanf("%d", &n);
  int processes[n];
  printf("Enter sizes of %d processes:\n", n);
  for (int i = 0; i < n; i++)
    scanf("%d", &processes[i]);

  firstFit(blocks, m, processes, n);
  bestFit(blocks, m, processes, n);
  worstFit(blocks, m, processes, n);

  return 0;
}
```

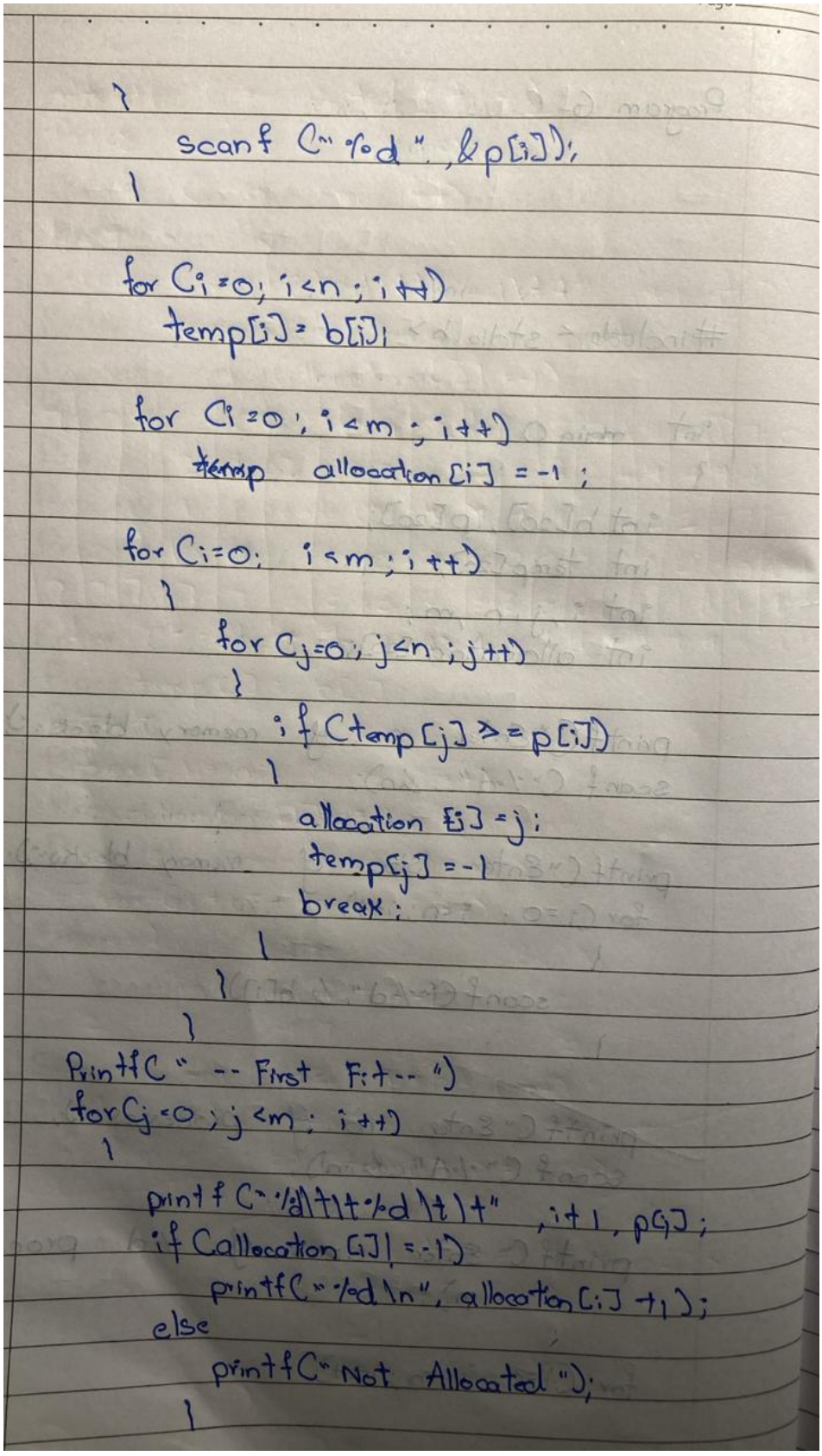
Result:

```
Name: PRATHEEK P REDDY
USN:1WA24CS215

Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.     Process Size    Block No.
1               212             2
2               417             5
3               112             3
4               426             Not Allocated

--- Best Fit ---
Process No.     Process Size    Block No.
1               212             4
2               417             2
3               112             3
4               426             5

--- Worst Fit ---
Process No.     Process Size    Block No.
1               212             5
2               417             2
3               112             4
4               426             Not Allocated


...Program finished with exit code 0
Press ENTER to exit console.
```

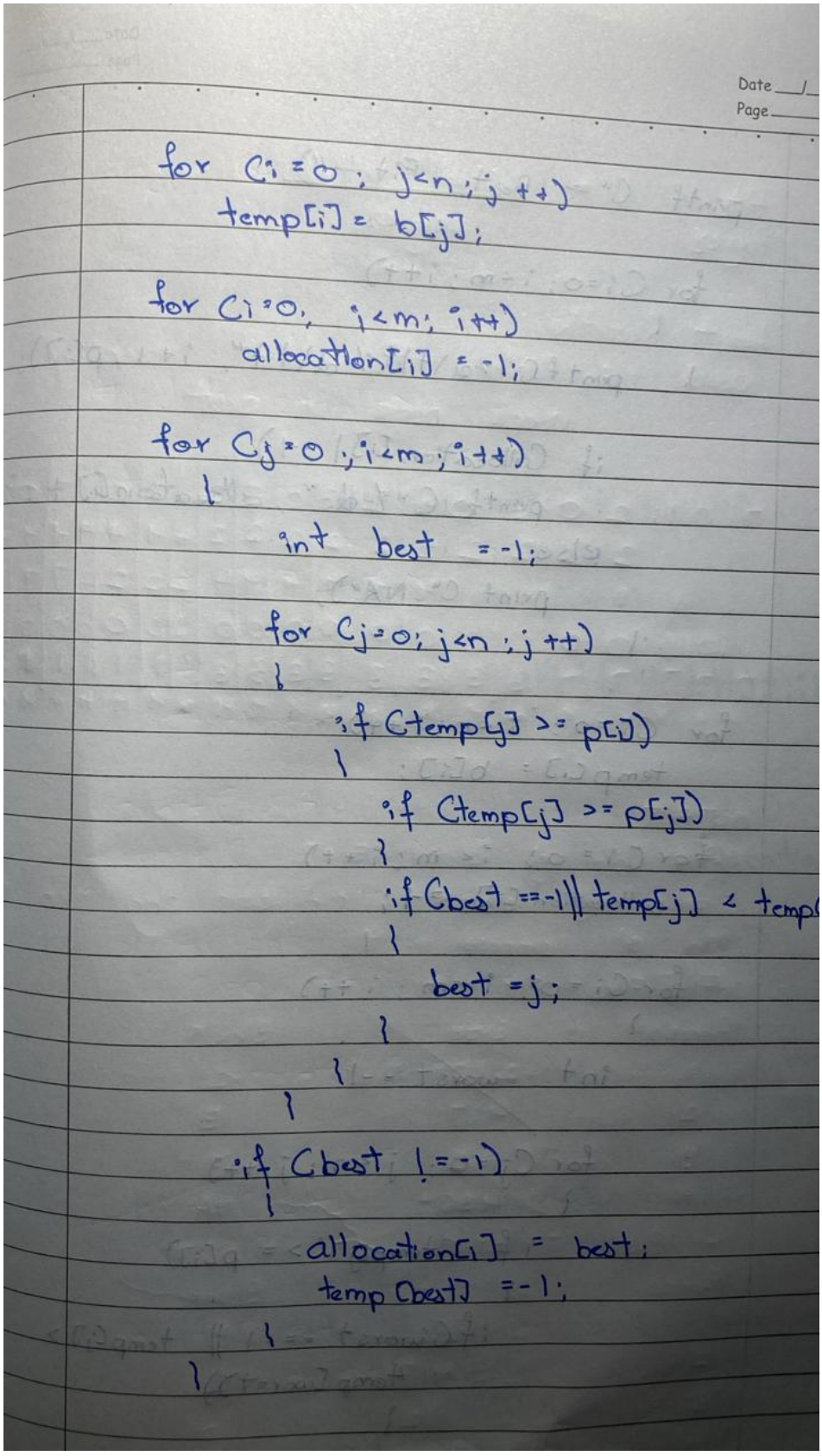
Program 6



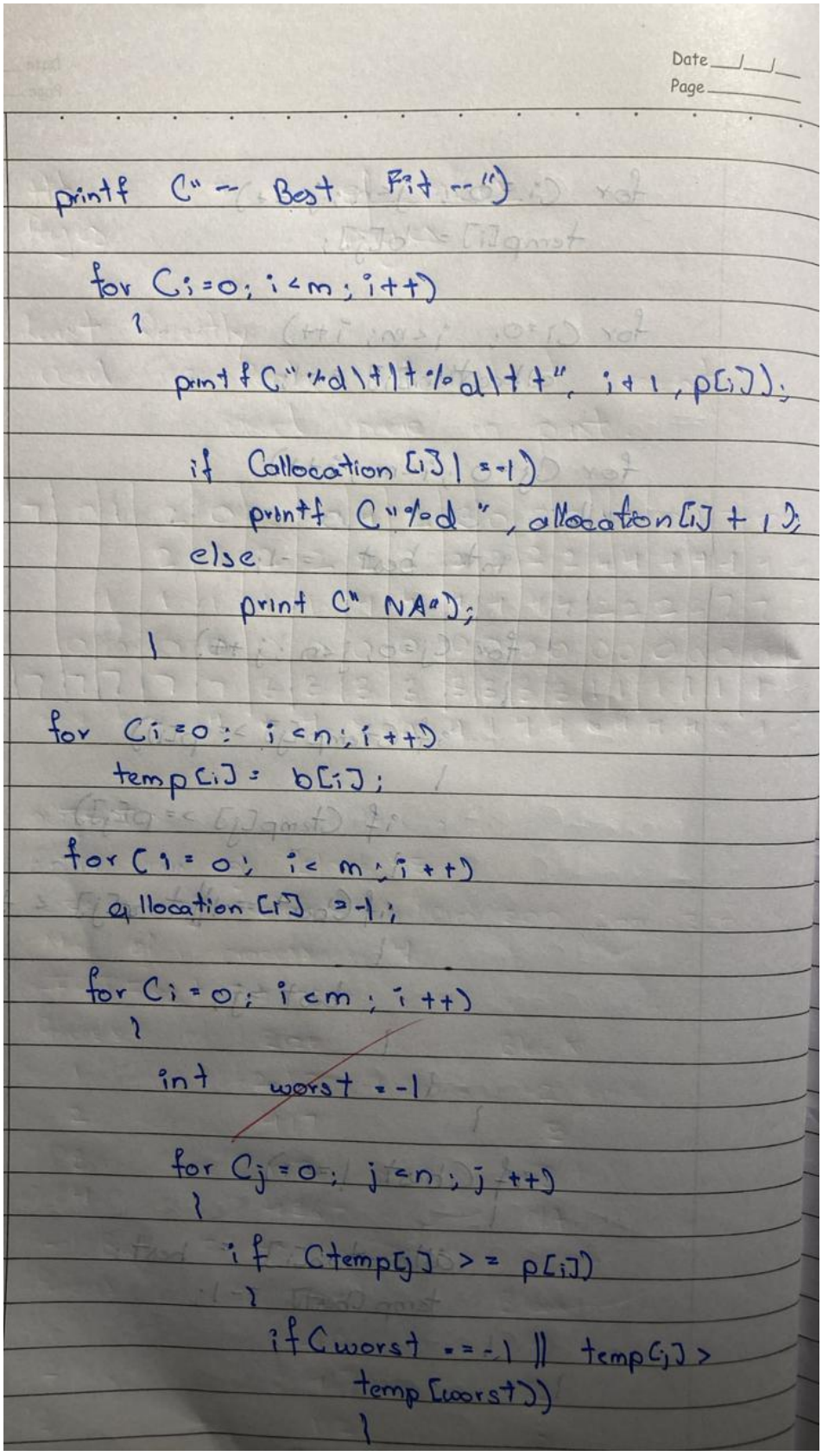
```
#include <stdio.h>

int main()
{
    int b[20], p[20];
    int temp[20];
    int i, j, n, m;
    int allocation[20];

    printf("Enter no. of memory blocks: ");
    scanf("%d", &n);

    printf("Enter size of memory blocks:");
    for(i=0; i<n; i++)
    {
        scanf("%d", &b[i]);
    }

    printf("Enter no. of processes: ");
    scanf("%d", &m);

    printf("Enter sizes of %d processes
        m);

    for(i=0; i<m; i++)
```



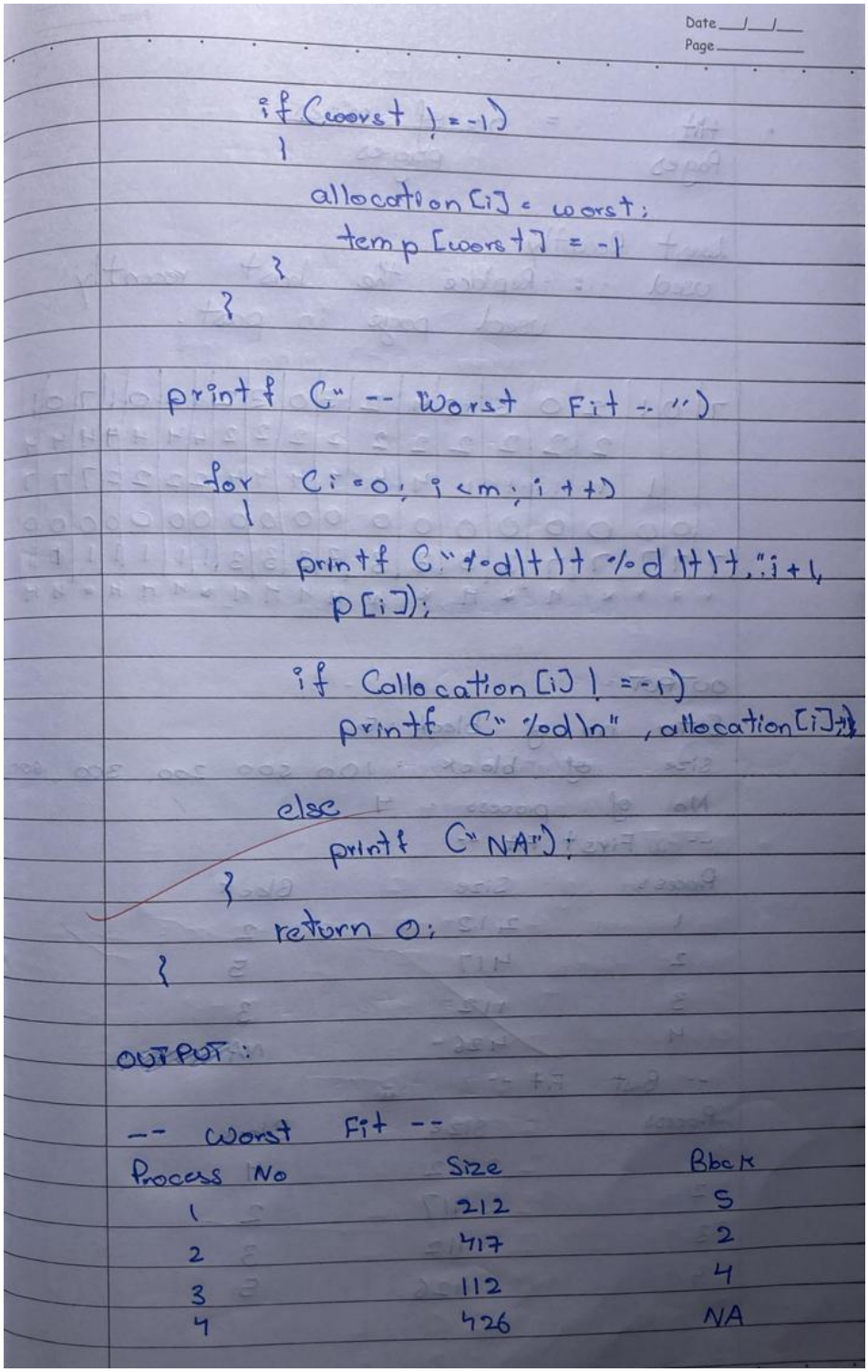
```
}
    scanf ("%d", &p[i]);
}

for (i=0; i<n; i++)
  temp[i] = b[i];

for (i=0; i<m; i++)
    temp  allocation[i] = -1;

for (i=0;  i<m; i++)
  {
    for (j=0; j<n; j++)
    {
      if (temp[j] >= p[i])
      {
        allocation[i] = j;
        temp[j] = -1
        break;
      }
    }
  }
Printf(" -- First Fit -- ")
for(j=0; j<m; i++)
{
  printf("%d\t\t%d\t\t", i+1, p[j];
  if (allocation[i] != -1)
    printf("%d\n", allocation[i]+1);
  else
    printf("Not Allocated");
}
```



```
for (i=0 ; j<n ; j ++)
    temp[i] = b[j];

for (i=0;  j<m; i++)
    allocation[i] = -1;

for (j=0 ; i<m ; i++)
{
    int best = -1;

    for (j=0; j<n ; j ++)
    {
        if (temp[j] >= p[i])
        {
            if (temp[j] >= p[j])
            }
            if (best == -1 || temp[j] < temp
            {
                best = j;
            }
        }
    }

if (best != -1)
{
    allocation[i] = best;
    temp [best] = -1;
}
}
```



```
printf ("-- Best Fit --")

for (i=0; i<m; i++)
{
    printf ("%d\t\t%d\t\t", i+1, p[i]);

    if (allocation[i] != -1)
        printf ("%d", allocation[i] + 1);
    else
        print ("NA");
}

for (i=0; i<n; i++)
    temp[i] = b[i];

for (i = 0; i < m; i++)
    allocation[i] = -1;

for (i=0; i<m; i++)
{
    int worst = -1

    for (j=0; j<n; j++)
    {
        if (temp[j] >= p[i])
        {
            if (worst == -1 || temp[j] >
                temp[worst]))
            {
```

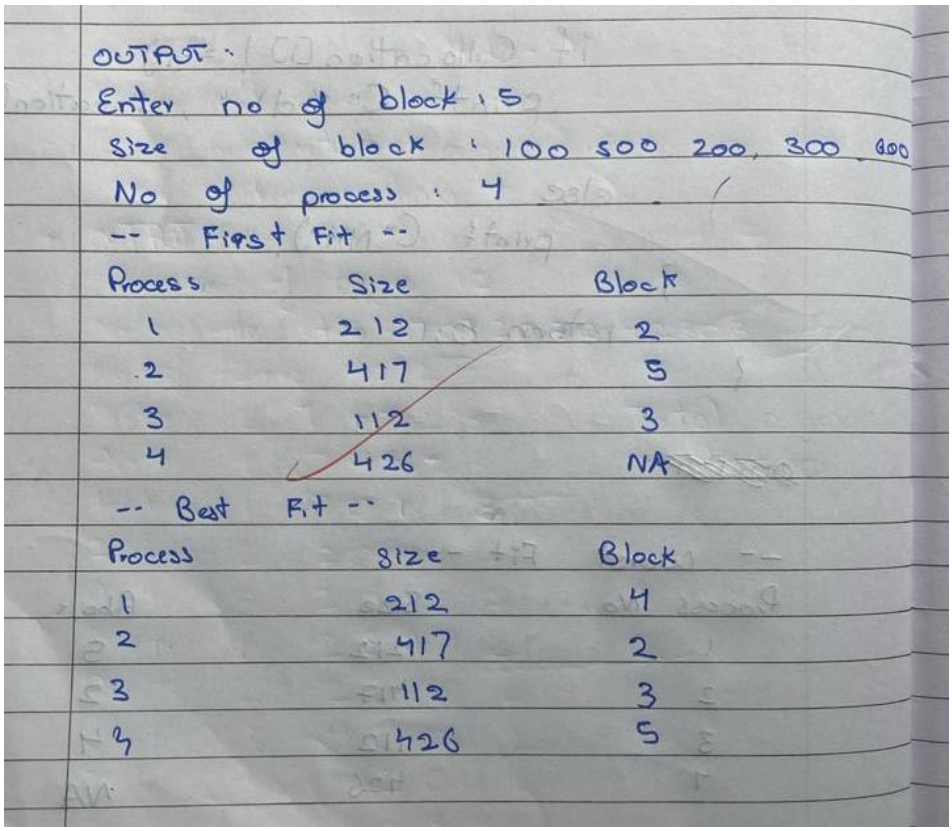


```
        if (worst != -1)
        {
            allocation[i] = worst;
            temp[worst] = -1;
        }
    }

    printf("\n-- Worst Fit --\n");
    for (i=0; i<m; i++)
    {
        printf("%d\t\t%d\t\t", i+1, p[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i]+1);
        else
            printf("NA");
    }
    return 0;
}
```

OUTPUT :

-- Worst Fit --

Process No	Size	Block
1	212	5
2	417	2
3	112	4
4	426	NA



OUTPUT:

Enter no of block : 5

Size of block : 100 500 200 300 600

No of process : 4

-- First Fit --

Process	Size	Block
1	212	2
2	417	5
3	112	3
4	426	NA

-- Best Fit --

Process	Size	Block
1	212	4
2	417	2
3	112	3
4	426	5

Program -7

Question:

Write a C program to simulate page replacement algorithms. (Any one)
a) FIFO
b) LRU
c) Optimal

FIFO
CODE:

```
#include <stdio.h>

int main() {

   printf("Name: PRATHEEK P REDDY\nUSN: 1WA24CS215\n\n");

   int pages[20], frames[10];
   int n, f, i, j, k = 0, fault = 0, found;

   printf("Enter number of pages: ");
   scanf("%d", &n);

   printf("Enter page reference string: ");
   for(i = 0; i < n; i++) {
      scanf("%d", &pages[i]);
   }

   printf("Enter number of frames: ");
   scanf("%d", &f);

   for(i = 0; i < f; i++) {
      frames[i] = -1;
   }
```

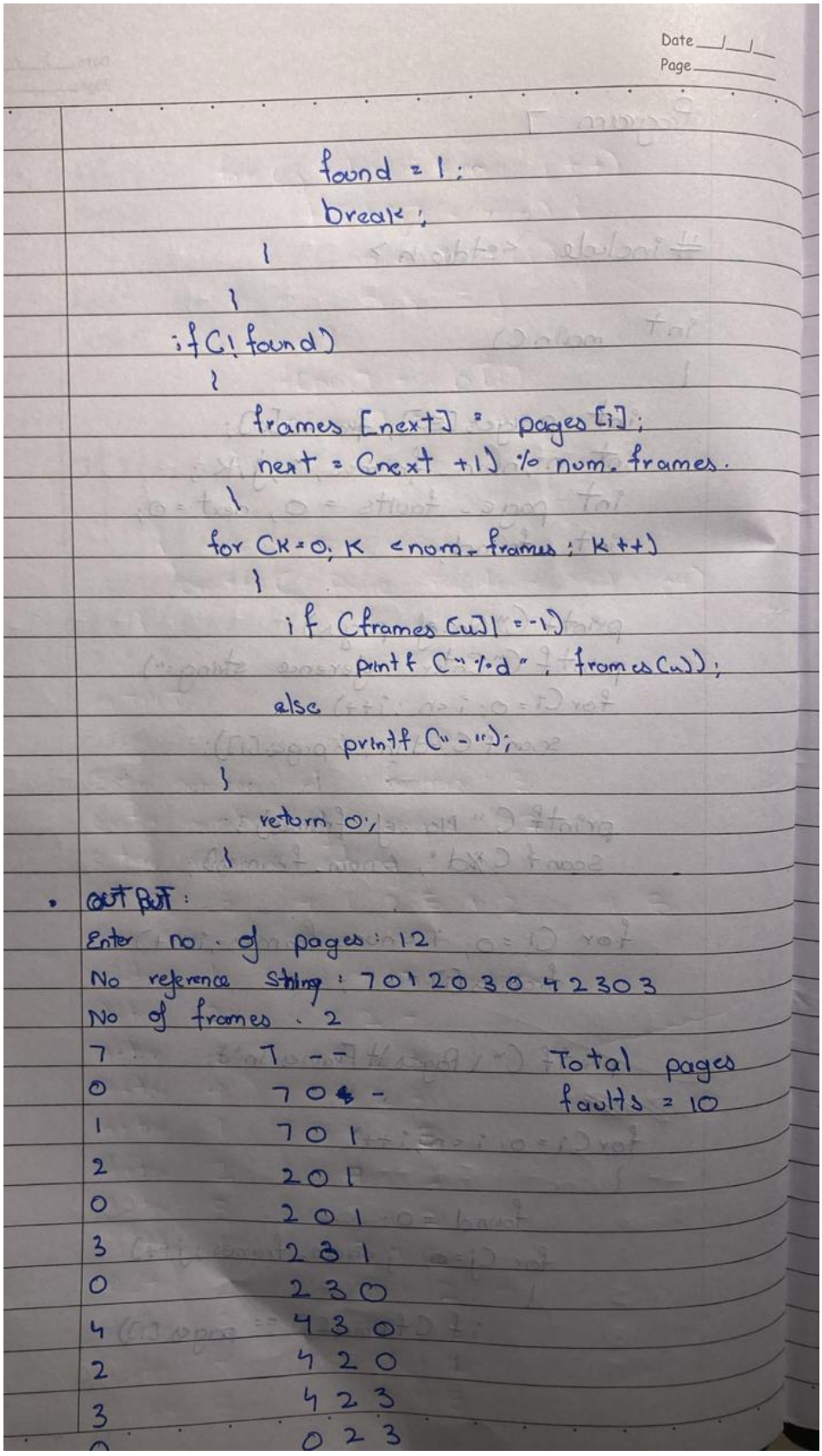
```
    for(i = 0; i < n; i++) {

        found = 0;

        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        if(found == 0) {
            frames[k] = pages[i];
            k = (k + 1) % f;
            fault++;
        }

        printf("\nPage %d -> ", pages[i]);

        for(j = 0; j < f; j++) {
            if(frames[j] != -1)
                printf("%d ", frames[j]);
            else
                printf("- ");
        }
    }

    printf("\n\nTotal Page Faults = %d\n", fault);

    return 0;
}
```

Result:

```
Name: PRATHEEK P REDDY
USN: 1WA24CS215

Enter number of pages: 12
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 2

Page 7 -> 7 -
Page 0 -> 7 0
Page 1 -> 1 0
Page 2 -> 1 2
Page 0 -> 0 2
Page 3 -> 0 3
Page 0 -> 0 3
Page 4 -> 4 3
Page 2 -> 4 2
Page 3 -> 3 2
Page 0 -> 3 0
Page 3 -> 3 0

Total Page Faults = 10


...Program finished with exit code 0
Press ENTER to exit console.
```

Program 7

```
#include <stdio.h>

int main()
{
    int pages[50], frames[10];
    int n, num_frames, i, j, k;
    int page_faults = 0, next = 0;
    int found;

    printf("No. of pages");
    printf("Enter reference string:")
    for(i = 0; i < n ; i++)
        scanf("%d\n", &pages[i]);

    printf("No. of frames");
    scanf("%d", &num_frames);

    for(i = 0; i < num_frames; i++)
        frames[i] = -1;

    printf("\nPages\tFrames\n");

    for(i = 0; i < n; i++)
    {
        found = 0;
        for(j = 0; j < num_frames; j++)
        {
            if(frames[j] == pages[i])
            {
```



```
                found = 1;
                break;
            }
        }
        if(!found)
        {
            frames[next] = pages[i];
            next = (next + 1) % num_frames;
        }
        for(k=0; k<num_frames; k++)
        {
            if(frames[j] != -1)
                printf("%d", frames[j]);
            else
                printf("-");
        }
        return 0;
    }
```

- OUTPUT:

```
Enter no. of pages: 12
No reference string: 7 0 1 2 0 3 0 4 2 3 0 3
No of frames: 2
```

7	7 -	Total pages faults = 10
0	7 0	
1	7 0 1	
2	2 0 1	
0	2 0 1	
3	2 3 1	
0	2 3 0	
4	4 3 0	
2	4 2 0	
3	4 2 3	
0	0 2 3	

LRU

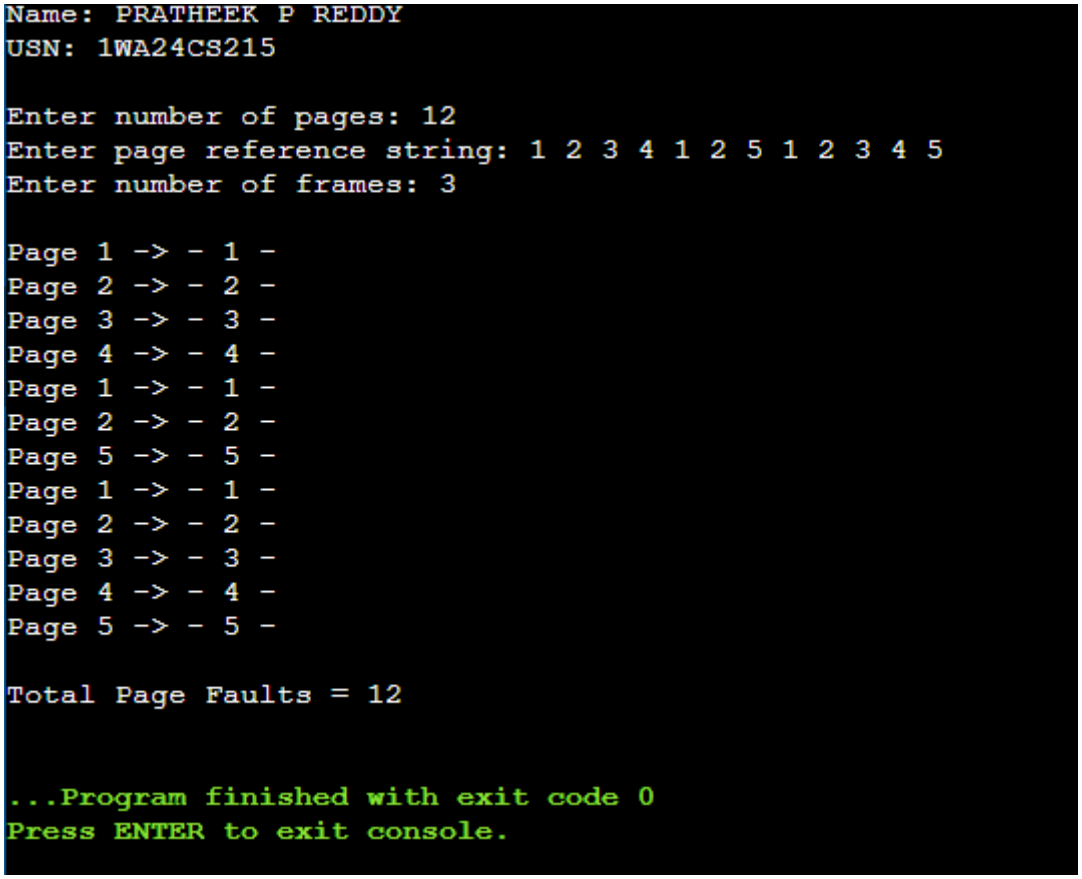
CODE:

```
#include <stdio.h>

int main() {

   printf("Name: PRATHEEK P REDDY\nUSN: 1WA24CS215\n\n");

   int pages[20], frames[10], time[10];
   int n, f, i, j, pos, fault = 0, count = 0, found;

   printf("Enter number of pages: ");
   scanf("%d", &n);

   printf("Enter page reference string: ");
   for(i = 0; i < n; i++) {
      scanf("%d", &pages[i]);
   }

   printf("Enter number of frames: ");
   scanf("%d", &f);

   for(i = 0; i < f; i++) {
      frames[i] = -1;
   }

   for(i = 0; i < n; i++) {

      found = 0;

      for(j = 0; j < f; j++) {
```

```
    if(frames[j] == pages[i]) {
        count++;
        time[j] = count;
        found = 1;
        break;
    }
}

if(found == 0) {

    pos = 0;

    for(j = 1; j < f; j++) {
        if(time[j] < time[pos]) {
            pos = j;
        }
    }

    frames[pos] = pages[i];
    count++;
    time[pos] = count;
    fault++;
}

printf("\nPage %d -> ", pages[i]);

for(j = 0; j < f; j++) {
    if(frames[j] != -1)
        printf("%d ", frames[j]);
    else
        printf("- ");
}
```

```
    }

    printf("\n\nTotal Page Faults = %d\n", fault);

    return 0;
}
```

Result:



OPTIMAL PAGE REPLACEMENT

CODE:

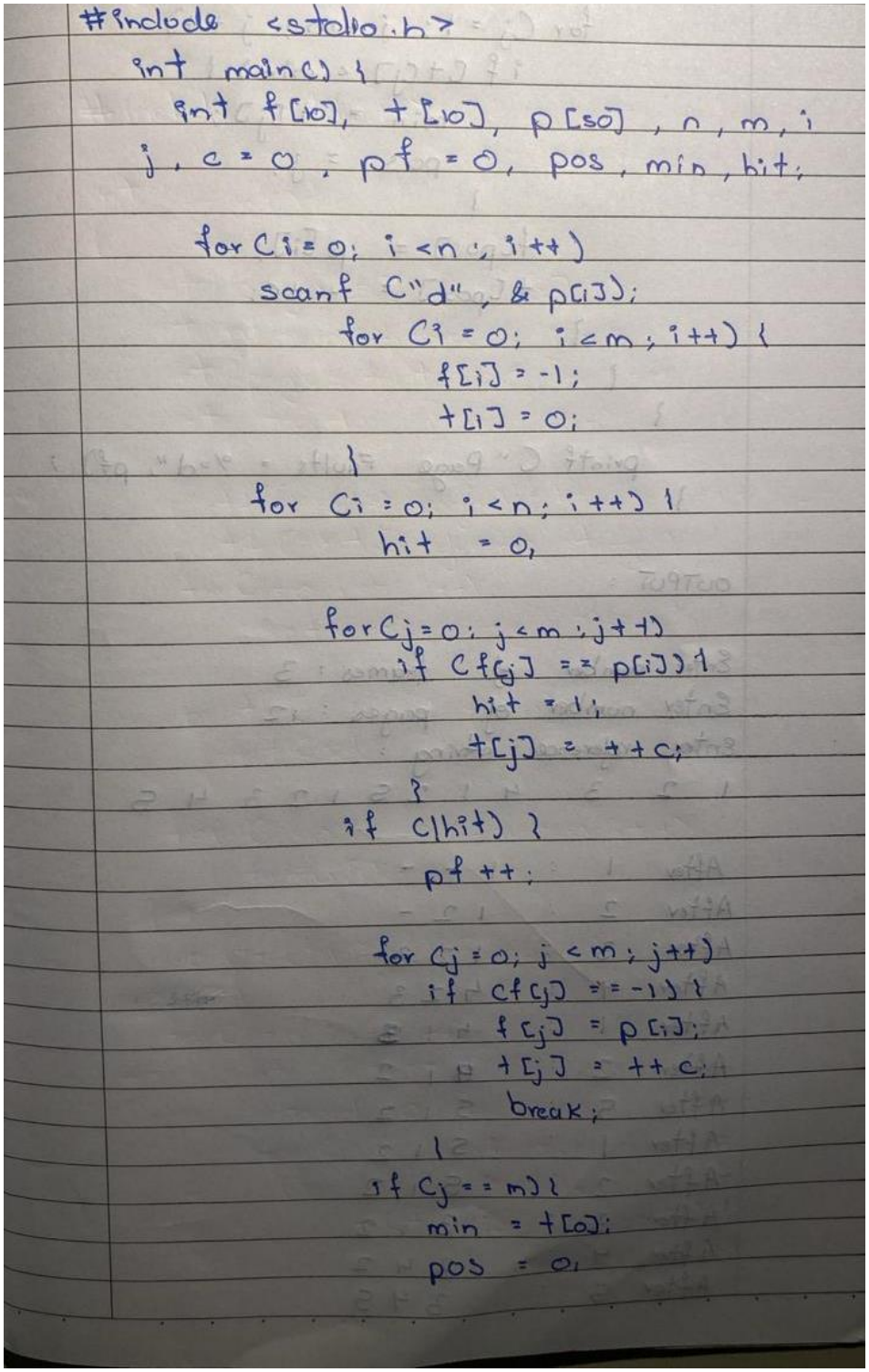
```
#include <stdio.h>

int main() {

   printf("Name: PRATHEEK P REDDY\nUSN: 1WA24CS215\n\n");

   int pages[20], frames[10];
   int n, f, i, j, k, pos, fault = 0, found;

   printf("Enter number of pages: ");
   scanf("%d", &n);

   printf("Enter page reference string: ");
   for(i = 0; i < n; i++) {
      scanf("%d", &pages[i]);
   }

   printf("Enter number of frames: ");
   scanf("%d", &f);

   for(i = 0; i < f; i++) {
      frames[i] = -1;
   }

   for(i = 0; i < n; i++) {

      found = 0;

      for(j = 0; j < f; j++) {
```

```
    if(frames[j] == pages[i]) {
        found = 1;
        break;
    }
}

if(found == 0) {

    pos = -1;

    for(j = 0; j < f; j++) {

        int future = 0;

        for(k = i + 1; k < n; k++) {
            if(frames[j] == pages[k]) {
                future = 1;
                break;
            }
        }

        if(future == 0) {
            pos = j;
            break;
        }
    }

    if(pos == -1)
        pos = 0;

    frames[pos] = pages[i];
    fault++;
```

```
        }

        printf("\nPage %d -> ", pages[i]);

        for(j = 0; j < f; j++) {
            if(frames[j] != -1)
                printf("%d ", frames[j]);
            else
                printf("- ");
        }
    }

    printf("\n\nTotal Page Faults = %d\n", fault);

    return 0;
}
```

Result:

```
Name: PRATHEEK P REDDY
USN: 1WA24CS215

Enter number of pages: 12
Enter page reference string: 1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Page 1 -> 1 - -
Page 2 -> 1 2 -
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 1 2 3
Page 2 -> 1 2 3
Page 5 -> 5 2 3
Page 1 -> 1 2 3
Page 2 -> 1 2 3
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 5 -> 5 2 3

Total Page Faults = 9
```

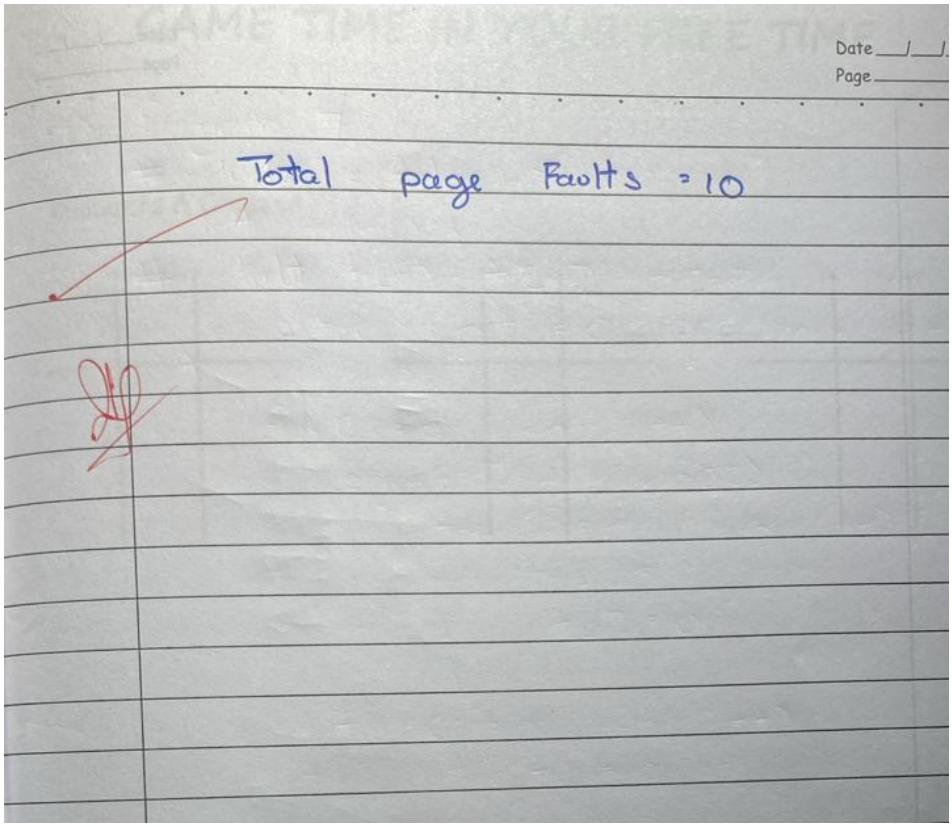


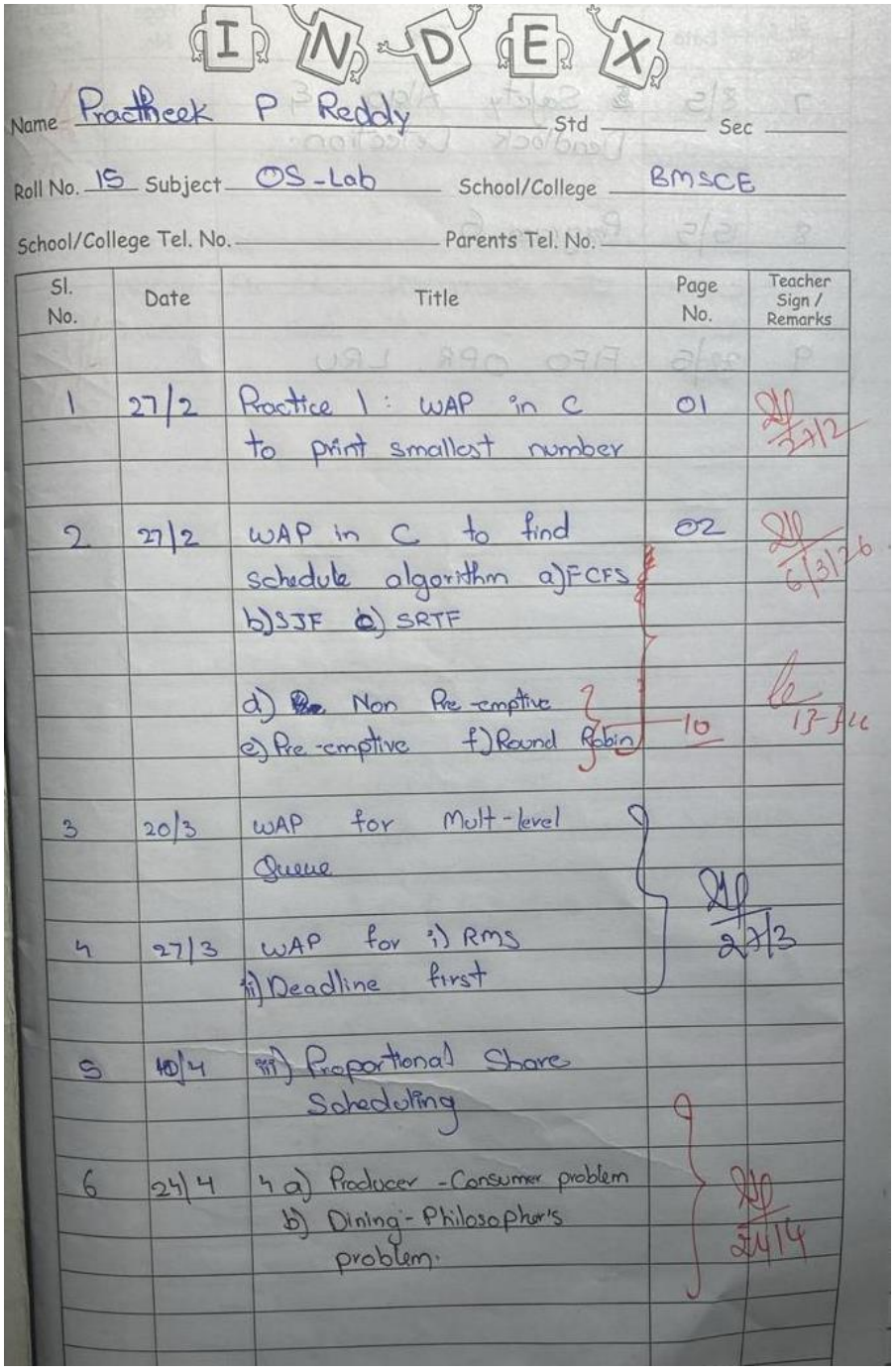
```
#include <stdio.h>
int main() {
  int f[10], t[10], p[50], n, m, i
  j, c = 0, pf = 0, pos, min, hit;

  for(i = 0; i < n; i++)
    scanf("%d", &p[i]);
      for(i = 0; i < m; i++) {
        f[i] = -1;
        t[i] = 0;
      }
    for(i = 0; i < n; i++) {
      hit = 0;

    for(j = 0; j < m; j++)
      if(f[j] == p[i]) {
        hit = 1;
        t[j] = ++c;
      }
    if(!hit) {
      pf++;

    for(j = 0; j < m; j++)
      if(f[j] == -1) {
        f[j] = p[i];
        t[j] = ++c;
        break;
      }
    if(j == m) {
      min = t[0];
      pos = 0;
```

```
        for (j = 1; j < m; j++)
            if (t[j] < min) {
                min = t[j];
                pos = j;
            }
        t[pos] = p[i];
        t[pos] = ++c;
    }
  }
  printf ("Page Faults = %d", pf);
}
```

OUTPUT :

```
Enter number of frames : 3
Enter number of pages : 12
Enter reference string :
1 2 3 4 1 2 5 1 2 3 4 5

After 1 : 1 - -
After 2 : 1 2 -
After 3 : 1 2 3
After 4 : 4 2 3
After 1 : 4 1 3
After 2 : 4 1 2
After 5 : 5 1 2
After 1 : 5 1 2
After 2 : 5 1 2
After 3 : 3 1 2
After 4 : 3 4 2
After 5 : 3 4 5
```


Date
Page
Total page Faults = 10



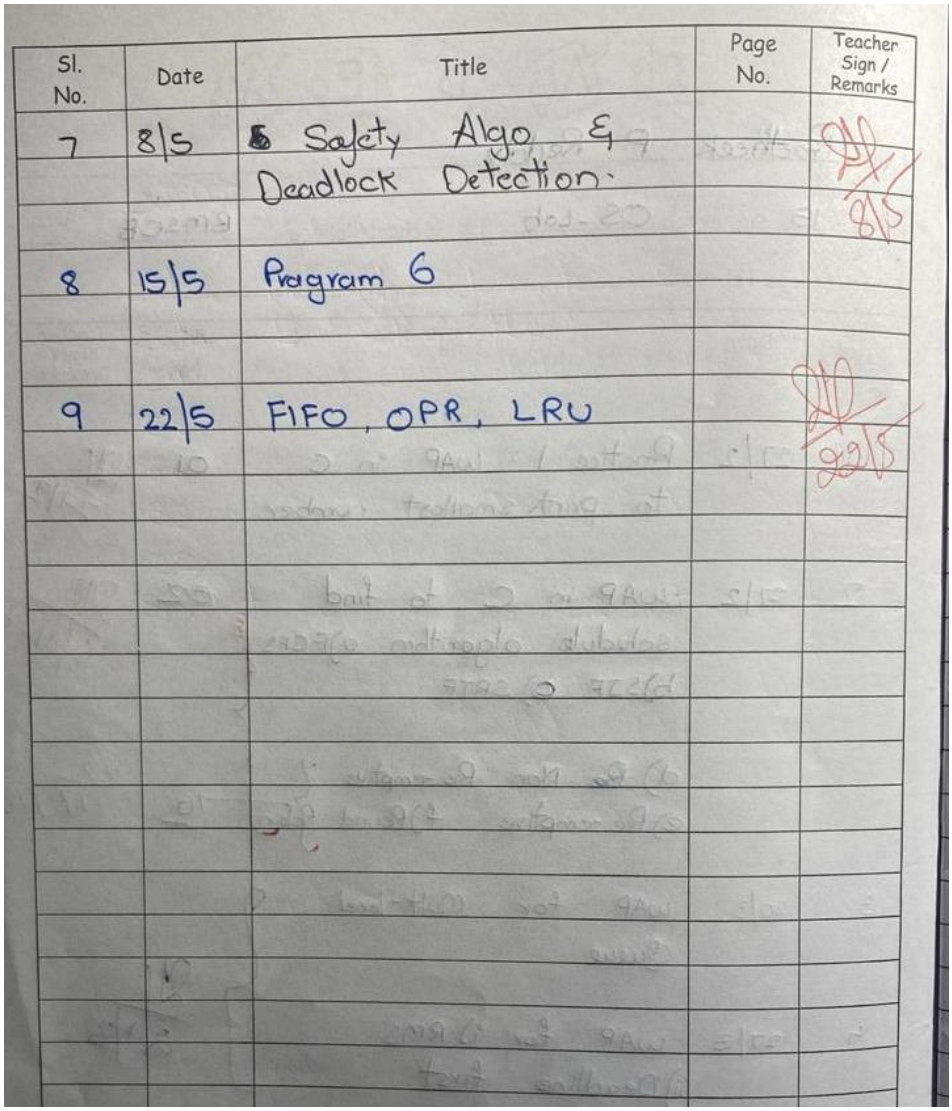
INDEX

Name Prathheek P Reddy Std Sec

Roll No. 15 Subject OS-Lab School/College BMSCE

School/College Tel. No. Parents Tel. No.

Sl. No.	Date	Title	Page No.	Teacher Sign / Remarks
1	27/2	Practice 1: WAP in C to print smallest number	01	27/2
2	27/2	WAP in C to find schedule algorithm a) FCFS b) SJF c) SRTF d) Non Pre-emptive e) Pre-emptive f) Round Robin	02 – 10	6/3/26 13-Mar
3	20/3	WAP for Mult-level Queue		27/3
4	27/3	WAP for i) RMS ii) Deadline first		
5	10/4	iii) Proportional Share Scheduling		
6	24/4	4 a) Producer - Consumer problem b) Dining - Philosopher's problem.		24/4



Sl. No.	Date	Title	Page No.	Teacher Sign / Remarks
7	8/5	Safety Algo & Deadlock Detection.		
8	15/5	Program 6		
9	22/5	FIFO, OPR, LRU		